



UNIVERSIDADE
VILA VELHA
ESPIRITO SANTO

DATA SCIENCE PROJECT

WORKFLOW PARTE 1 DO PROJETO

NOME COMPLETO: DAVI VAGO SOARES

LINK PARA SEU PRÓPRIO COLAB: [davi_vago_soares_WORKFLOW_PARTE_1](#)

LINK PARA SEU PRÓPRIO GITHUB: [uvv-wine](#)

LINK PARA SEU VÍDEO NO YOUTUBE: [Aqui](#)



UWine – Clube de Vinhos da UVV



Cenário Fictício:

Prezados(as) alunos(as),

É com entusiasmo que apresento a proposta do **Projeto de Conclusão de Curso (TCC)** para os estudantes do curso de Ciência de Dados da Universidade Vila Velha: o UWine – Clube de Vinhos da UVV.

A proposta do projeto consiste na análise de um Clube de Vinhos universitário fictício, com base em dados simulados, que será totalmente orientado por técnicas e práticas de Ciência de Dados.

Sendo assim, você foi designado como **Consultor em Ciência de Dados (Data Science Consultant)** para fazer uma análise completa dos dados dos clientes a partir de um conjunto de dados (Amostra) com mais de 1 milhão de notas fiscais (~ 350 Megabytes). Além disso, há uma entrada diária de aproximadamente umas 5000 novas notas fiscais no sistema.

O objetivo é permitir que cada consultor(a) percorra foco analíticos diferentes dentro do mesmo ecossistema de dados, aplicando modelos de machine learning supervisionado ou não supervisionado, análise preditiva, dashboards interativos, entre outros.

Para garantir organização, profundidade técnica e coerência metodológica no desenvolvimento do **Projeto UWine**, adotaremos um workflow composto por cinco etapas principais, alinhadas às boas práticas da Ciência de Dados. Cada etapa representa um marco do processo analítico, permitindo que cada analista avance de forma estruturada do entendimento inicial dos dados até a entrega final dos insights.

BIBLIOTECAS E FUNÇÕES E CÓDIGOS

```
In [1]: # Manipulação de Dados
import pandas as pd
import numpy as np

# Visualização de Dados
import matplotlib.pyplot as plt
import seaborn as sns

# Estatística
from scipy import stats
from scipy.stats import (
    pearsonr,
    anderson,
    ttest_ind
)

# Machine Learning
from sklearn.model_selection import train_test_split
```

```
In [ ]: # Load do seu TABLE7 aqui: Numero 07
df = pd.read_csv('table7.csv', index_col=0)
```

```
In [ ]: df.head()
```

Out[]:

	NOTA FISCAL	REGIÃO	SEXO	ESTADO CIVIL	DEPENDENTES	RENDA BRUTO (R\$)	OPINIÃO DO CLIENTE
0	3788072	NORDESTE	MASCULINO	CASADO	0	3226.55	Gostei atendime presta pela UVV
1	2938466	SUDESTE	FEMININO	SOLTEIRO	3	7560.73	Não tem muit dizer sol a UVV
2	3349617	NORTE	MASCULINO	DESQUITADO(A) / VIÚVO(A)	3	3306.75	Serv eficier recomen o clu UVV
3	3966132	CENTRO-OESTE	MASCULINO	CASADO	0	7649.26	Gostei atendime presta pela UVV
4	4915368	NORDESTE	MASCULINO	CASADO	1	3110.09	Serv impecá adore clu UVV

5 rows × 23 columns

```
In [ ]: df.shape
```

Out[]: (1120000, 23)

```
In [ ]: df.dtypes
```

Out[]:

0

NOTA FISCAL	int64
REGIÃO	object
SEXO	object
ESTADO CIVIL	object
DEPENDENTES	int64
RENDAMENTO BRUTO (R\$)	float64
OPINIÃO DO CLIENTE	object
NOTA DE SATISFAÇÃO (%)	float64
MEAN: alcohol	float64
MEAN: malic_acid	float64
MEAN: ash	float64
MEAN: alcalinity_of_ash	float64
MEAN: magnesium	float64
MEAN: total_phenols	float64
MEAN: flavanoids	float64
MEAN: nonflavanoid_phenols	float64
MEAN: proanthocyanins	float64
MEAN: color_intensity	float64
MEAN: hue	float64
MEAN: od280/od315_of_diluted_wines	float64
MEAN: proline	float64
TIPO DA CONTA	object
TOTAL (R\$)	float64

dtype: object

In []: `df.isnull().sum()`

```
Out[ ]:
```

	0
NOTA FISCAL	0
REGIÃO	0
SEXO	0
ESTADO CIVIL	0
DEPENDENTES	0
RENDIA BRUTO (R\$)	0
OPINIÃO DO CLIENTE	0
NOTA DE SATISFAÇÃO (%)	0
MEAN: alcohol	0
MEAN: malic_acid	0
MEAN: ash	0
MEAN: alkalinity_of_ash	0
MEAN: magnesium	0
MEAN: total_phenols	0
MEAN: flavanoids	0
MEAN: nonflavanoid_phenols	0
MEAN: proanthocyanins	0
MEAN: color_intensity	0
MEAN: hue	0
MEAN: od280/od315_of_diluted_wines	0
MEAN: proline	0
TIPO DA CONTA	0
TOTAL (R\$)	0

```
dtype: int64
```

Atividade A1

Atividade A1.1

```
In [ ]: def AtividadeA1_1(df, coluna_gastos='TOTAL (R$)', coluna_tipo='TIPO DA CONTA'):
```

```

    """
    Calcula e plota o erro padrão para diferentes tamanhos de amostra (400 a 200
    Gera 4 gráficos: um geral e um para cada tipo de conta. Retorna dicionário c
    """

    # Gráfico geral
```

```

array_se_geral = np.zeros(500, dtype=float)
j = 0
for i in range(400, 200_400, 400):
    tb = df.sample(n=i, replace=False)[coluna_gastos]
    array_se_geral[j] = tb.std() / np.sqrt(len(tb))
    j += 1

# Gráficos por tipo de conta
tipos_conta = df[coluna_tipo].unique()
arrays_se_tipos = {}

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

# Gráfico geral
x = np.arange(1, 501, 1)
axes[0].plot(x, array_se_geral)
axes[0].set_title(f"GERAL - Erro-Padrão Médio: R${np.mean(array_se_geral):.2}")
axes[0].set_ylabel('ERRO PADRÃO (R$)')
axes[0].set_xlabel('TAMANHO DA AMOSTRA * 400')
axes[0].hlines(np.mean(array_se_geral), 0, 500, colors='red', linestyle='--')
axes[0].grid(True, alpha=0.3)

# Gráfico para cada tipo de conta
for idx, tipo in enumerate(tipos_conta, start=1):
    df_tipo = df[df[coluna_tipo] == tipo]
    tamanho_max = min(200_000, len(df_tipo))
    n_iteracoes = tamanho_max // 400

    array_se_tipo = np.zeros(n_iteracoes, dtype=float)
    j = 0
    for i in range(400, tamanho_max + 1, 400):
        tb = df_tipo.sample(n=i, replace=False)[coluna_gastos]
        array_se_tipo[j] = tb.std() / np.sqrt(len(tb))
        j += 1

    arrays_se_tipos[tipo] = array_se_tipo
    x_tipo = np.arange(1, len(array_se_tipo) + 1, 1)
    axes[idx].plot(x_tipo, array_se_tipo)
    axes[idx].set_title(f"{tipo} - Erro-Padrão Médio: R${np.mean(array_se_tipo):.2}")
    axes[idx].set_ylabel('ERRO PADRÃO (R$)')
    axes[idx].set_xlabel('TAMANHO DA AMOSTRA * 400')
    axes[idx].hlines(np.mean(array_se_tipo), 0, len(array_se_tipo), colors='red', linestyle='--')
    axes[idx].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

return {'geral': array_se_geral, **arrays_se_tipos}

```

Atividade A1.2

```

In [ ]: def Atividade1_2(table, reference_column):
    """
    DOCSTRING:
    table = Seu dataframe
    reference_column = Escolha a coluna que será
    a referência para o cálculo dos outliers.

```

```

Ex. de Sintaxe: Atividade1_2(my_table, my_table['Column Name'])
'''

q1 = reference_column.quantile(.25)
q3 = reference_column.quantile(.75)
iqr = q3 - q1
reference_limite_i = q1 - 1.5 * iqr ## limite inferior
reference_limite_s = q3 + 1.5 * iqr ## limite superior
table_etl = table[(reference_column > reference_limite_i) & (reference_column

return table_etl

```

Atividade A1.3.1

```

In [ ]: def Atividade1_3_1(data):
        """
        Executa o Teste de Normalidade Anderson-Darling.
        Exibe estatística do teste e interpretação por intervalo de confiança.
        """

        result = anderson(data)
        print("=" * 60)
        print("TESTE DE NORMALIDADE")
        print("=" * 60)
        print(f'DISTRIBUIÇÃO AMOSTRAL: stat = {result.statistic:.3f}')
        for i in range(len(result.critical_values)):
            sl, cv = result.significance_level[i], result.critical_values[i]
            if result.statistic < cv:
                print(f'Intervalo de Confiança ({sl:.1f}%): Probably Gaussian.')
            else:
                print(f'Intervalo de Confiança ({sl:.1f}%): Probably Not Gaussian.')

```

Atividade A1.3.2

```

In [ ]: def Atividade1_3_2(df, coluna1, coluna2):
        """
        Executa o Teste de Correlação de Pearson entre duas variáveis numéricas.
        Retorna coeficiente de correlação, p-valor e interpretação da independência.
        """

        coef_correlacao, p_valor = pearsonr(df[coluna1], df[coluna2])

        print(f'Coeficiente de Correlação de Pearson: {coef_correlacao:.4f}')
        print(f'P-valor: {p_valor:.6f}')
        print('\nInterpretação da Correlação:')

        if abs(coef_correlacao) < 0.3:
            print('Correlação FRACA')
        elif abs(coef_correlacao) < 0.7:
            print('Correlação MODERADA')
        else:
            print('Correlação FORTE')

        print('\nInterpretação da Independência:')
        alpha = 0.05
        if p_valor < alpha:
            print(f'P-valor ({p_valor:.6f}) < {alpha}: As variáveis são DEPENDENTES')
        else:
            print(f'P-valor ({p_valor:.6f}) >= {alpha}: As variáveis são INDEPENDENTES')

```

```
return {'coeficiente': coef_correlacao, 'p_valor': p_valor}
```

Atividade A1.3.3

```
In [ ]: def Atividade1_3_3(amostra1, amostra2, nome_amostra1='Amostra 1', nome_amostra2='Amostra 2'):
        """
        Executa o Teste t de Student para comparar médias de duas amostras independentes.
        Retorna estatística t, p-valor e interpretação se as médias são significativamente diferentes.
        """

        estatistica_t, p_valor = ttest_ind(amostra1, amostra2)

        print(f'Teste t de Student: {nome_amostra1} vs {nome_amostra2}')
        print(f'Estatística t: {estatistica_t:.4f}')
        print(f'P-valor: {p_valor:.6f}')
        print(f'Média {nome_amostra1}: {amostra1.mean():.2f}')
        print(f'Média {nome_amostra2}: {amostra2.mean():.2f}')
        print('\nInterpretação:')

        alpha = 0.05
        if p_valor < alpha:
            print(f'P-valor ({p_valor:.6f}) < {alpha}: As médias são SIGNIFICATIVAMENTE DIFERENTES')
        else:
            print(f'P-valor ({p_valor:.6f}) >= {alpha}: As médias NÃO são significativamente diferentes')

        return {'estatistica_t': estatistica_t, 'p_valor': p_valor}
```

Atividade A1.3.4

```
In [ ]: def Atividade1_3_4(df, coluna_valores, coluna_grupos):
        """
        Executa Teste ANOVA (paramétrico) ou Kruskal-Wallis (não-paramétrico) para comparar médias de múltiplas amostras.
        Retorna estatística, p-valor e interpretação se as médias dos grupos são diferentes.
        """

        from scipy.stats import f_oneway, kruskal
        import numpy as np

        # Separar dados por grupo
        grupos = df[coluna_grupos].unique()
        dados_grupos = [df[df[coluna_grupos] == grupo][coluna_valores].values for grupo in grupos]

        # Executar ANOVA (One-Way)
        estatistica_f, p_valor_anova = f_oneway(*dados_grupos)

        # Executar Kruskal-Wallis (alternativa não-paramétrica)
        estatistica_h, p_valor_kruskal = kruskal(*dados_grupos)

        print("=" * 60)
        print("TESTE DAS MÉDIAS AMOSTRAIS")
        print("=" * 60)
        print(f"Grupos testados: {list(grupos)}")
        print(f"\nMédias por grupo:")
        for grupo in grupos:
            media = df[df[coluna_grupos] == grupo][coluna_valores].mean()
            print(f"    {grupo}: R$ {media:.2f}")
```



```

print("\n" + "-" * 60)
print("ANOVA (Teste Paramétrico):")
print(f"  Estatística F: {estatistica_f:.4f}")
print(f"  P-valor: {p_valor_anova:.6f}")

print("\nKruskal-Wallis (Teste Não-Paramétrico):")
print(f"  Estatística H: {estatistica_h:.4f}")
print(f"  P-valor: {p_valor_kruskal:.6f}")

print("\n" + "-" * 60)
print("Interpretação:")
alpha = 0.05
if p_valor_kruskal < alpha:
    print(f"P-valor ({p_valor_kruskal:.6f}) < {alpha}: As médias dos grupos
else:
    print(f"P-valor ({p_valor_kruskal:.6f}) >= {alpha}: As médias dos grupos

return {
    'anova_f': estatistica_f,
    'anova_p_valor': p_valor_anova,
    'kruskal_h': estatistica_h,
    'kruskal_p_valor': p_valor_kruskal
}

```

Atividade A2

Atividade A2.1

```

In [ ]: def Atividade2_1(df, colunas_quantitativas=None):
        """
        Analisa e plota estatísticas descritivas para variáveis quantitativas.
        Exibe gráfico de linha com medidas estatísticas e destaca média, mediana e m
        """

        # Selecionar colunas quantitativas automaticamente se não especificadas
        if colunas_quantitativas is None:
            colunas_quantitativas = [
                coluna
                for coluna in df.select_dtypes(include=['int64', 'float64']).columns
                if coluna != 'NOTA FISCAL'
            ]

        n_colunas = len(colunas_quantitativas)
        n_cols = 2 # 2 gráficos por linha
        n_rows = int(np.ceil(n_colunas / n_cols))

        fig, axes = plt.subplots(n_rows, n_cols, figsize=(16, 5 * n_rows))
        axes = axes.flatten() if n_colunas > 1 else [axes]

        resultados = {}

        for idx, coluna in enumerate(colunas_quantitativas):
            dataset = df[coluna].dropna()

            # Calcular estatísticas
            desc = dataset.describe()
            moda = dataset.mode()[0] if len(dataset.mode()) > 0 else dataset.median(

```

```

# dicionário de estatísticas na ordem
estatisticas = {
    'min': desc['min'],
    'std': desc['std'],
    '25%': desc['25%'],
    'mode': moda,
    '50%': desc['50%'],
    'mean': desc['mean'],
    '75%': desc['75%'],
    'range': desc['max'] - desc['min'],
    'max': desc['max']
}

resultados[coluna] = {
    'count': desc['count'],
    'mean': desc['mean'],
    'median': desc['50%'],
    'mode': moda,
    'std': desc['std'],
    'min': desc['min'],
    'max': desc['max'],
    'Q1': desc['25%'],
    'Q3': desc['75%']
}

# Preparar Labels
mean_label = f'MÉDIA:  $\mu^-$  = {estatisticas["mean"]:.2f}'
median_label = f'MEDIANA:  $\mu^-$  = {estatisticas["50%"]:.2f}'
mode_label = f'MODA:  $\mu^-$  = {estatisticas["mode"]:.2f}'

# Plotar
ax = axes[idx]
indices = list(estatisticas.keys())
valores = list(estatisticas.values())

ax.plot(indices, valores, marker='o', linewidth=2, markersize=6, color='

# Destacar medidas de tendência central
ax.plot('mean', estatisticas['mean'], 'D', markersize=10,
        color='#ff7f0e', zorder=5)
ax.plot('50%', estatisticas['50%'], 'D', markersize=10,
        color='#2ca02c', zorder=5)
ax.plot('mode', estatisticas['mode'], 'D', markersize=10,
        color='#d62728', zorder=5)

# Criar Legenda customizada
legend_elements = [
    plt.Line2D([0], [0], marker='D', color='w', markerfacecolor='#ff7f0e',
               markersize=8, label=mean_label),
    plt.Line2D([0], [0], marker='D', color='w', markerfacecolor='#2ca02c',
               markersize=8, label=median_label),
    plt.Line2D([0], [0], marker='D', color='w', markerfacecolor='#d62728',
               markersize=8, label=mode_label)
]

ax.set_title(f'ESTATÍSTICA DESCRITIVA DE DADOS - {coluna}',
             fontsize=11, fontweight='bold')
ax.set_ylabel(coluna, fontsize=10)
ax.set_xlabel('Medidas Estatísticas', fontsize=10)

```

```

ax.legend(handles=legend_elements, title='MEDIDAS DE RESUMO:',
          loc='best', fontsize=9)
ax.grid(True, alpha=0.3, linestyle='--')

# Remover subplots extras
for idx in range(n_colunas, len(axes)):
    fig.delaxes(axes[idx])

plt.tight_layout()
plt.show()

# Exibir tabela consolidada
df_resultados = pd.DataFrame(resultados).T
print("\n" + "=" * 100)
print("ESTATÍSTICAS DESCRITIVAS - VARIÁVEIS QUANTITATIVAS")
print("=" * 100)
print(df_resultados.to_string())
print("=" * 100)

return resultados

```

Atividade A2.2

```

In [ ]: def Atividade2_2(df, colunas_qualitativas=None):
        """
        Analisa e plota estatísticas descritivas para variáveis qualitativas (categó
        Exibe gráficos de barras com frequências absolutas e relativas para cada var
        """

        if colunas_qualitativas is None:
            colunas_qualitativas = [
                coluna
                for coluna in df.select_dtypes(include=['object']).columns
            ]

        n_colunas = len(colunas_qualitativas)
        n_cols = 2 # 2 gráficos por linha
        n_rows = int(np.ceil(n_colunas / n_cols))

        fig, axes = plt.subplots(n_rows, n_cols, figsize=(16, 5 * n_rows))
        axes = axes.flatten() if n_colunas > 1 else [axes]

        resultados = {}

        for idx, coluna in enumerate(colunas_qualitativas):
            # Calcular frequências
            freq_abs = df[coluna].value_counts()
            freq_rel = df[coluna].value_counts(normalize=True) * 100

            # Armazenar resultados
            resultados[coluna] = pd.DataFrame({
                'Frequência Absoluta': freq_abs,
                'Frequência Relativa (%)': freq_rel
            })

            # Plotar gráfico de barras
            ax = axes[idx]
            bars = ax.bar(range(len(freq_abs)), freq_abs.values,
                          color='#2E86AB', edgecolor='black', alpha=0.8)

```

```

# Adicionar valores sobre as barras
for i, (bar, valor) in enumerate(zip(bars, freq_abs.values)):
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2., height,
            f'{int(valor):,}\n({freq_rel.values[i]:.1f}%)',
            ha='center', va='bottom', fontsize=9, fontweight='bold')

ax.set_title(f'Distribuição - {coluna}', fontsize=12, fontweight='bold')
ax.set_xlabel('Categorias', fontsize=10)
ax.set_ylabel('Frequência Absoluta', fontsize=10)
ax.set_xticks(range(len(freq_abs)))
ax.set_xticklabels(freq_abs.index, rotation=45, ha='right')
ax.grid(True, alpha=0.3, axis='y')

# Remover subplots extras
for idx in range(n_colunas, len(axes)):
    fig.delaxes(axes[idx])

plt.tight_layout()
plt.show()

# Exibir tabelas de frequência
print("\n" + "=" * 100)
print("ESTATÍSTICAS DESCRITIVAS - VARIÁVEIS QUALITATIVAS")
print("=" * 100)
for coluna, tabela in resultados.items():
    print(f"\n{coluna}:")
    print("-" * 100)
    print(tabela.to_string())
print("\n" + "=" * 100)

return resultados

```

Atividade A3

Atividade A3.1

```

In [ ]: def Atividade3_1(df, coluna_opiniao='OPINIÃO DO CLIENTE'):
        """
        Discretiza a variável 'OPINIÃO DO CLIENTE' em categorias: Positiva, Neutra,
        Retorna o dataframe com nova coluna 'OPINIÃO DISCRETIZADA' e exibe distribui
        """

        # Dicionário de classificação das opiniões
        classificacao = {
            # MUITO SATISFEITO
            'Serviço impecável, adorei o clube: UVVine': 'Muito Satisfeito',
            'Superou expectativas o atendimento na UVVine': 'Muito Satisfeito',
            'Ótimo! Tudo ocorreu maravilhosamente bem com a UVVine': 'Muito Satisfe
            'Produto perfeito, chegou antes do prazo de previsão. Top demais a UVVin
            'Excelente experiência, recomendo muito a UVVine': 'Muito Satisfeito',

            # SATISFEITO
            'Entrega foi boa e rápida na UVVine': 'Satisfeito',
            'Funcionou bem e me atendeu no prazo certo a UVVine': 'Satisfeito',
            'Fiquei satisfeito com o serviço da UVVine': 'Satisfeito',

```

```

'Serviço eficiente, recomendo o clube: UVVine': 'Satisfeito',
'Gostei do atendimento prestado pela UVVine': 'Satisfeito',

# INDIFERENTE
'Nada digno de nota com a UVVine': 'Indiferente',
'Foi indiferente minha experiência com a UVVine': 'Indiferente',
'Não tenho muito a dizer sobre a UVVine': 'Indiferente',
'Sem opinião formada sobre a UVVine': 'Indiferente',
'Compro exponencialmente, opinião neutra sobre UVVine': 'Indiferente',

# INSATISFEITO
'Atendimento comum, poderia melhorar em cordialidade a UVVine': 'Insatisfeito',
'Nada demais, serviço regular na UVVine': 'Insatisfeito',
'Experiência regular, não atendeu a política de cashback da UVVine': 'Insatisfeito',
'Razoável, com margem de melhora no preço médio da UVVine': 'Insatisfeito',
'Foi aceitável, esperava mais da UVVine': 'Insatisfeito',
'Senti que faltou qualidade no atendimento da UVVine': 'Insatisfeito',
'Tive uma experiência ruim com a UVVine': 'Insatisfeito',
'Não gostei do atendimento na UVVine': 'Insatisfeito',
'Serviço deixou a desejar na UVVine': 'Insatisfeito',
'Produto chegou atrasado na UVVine': 'Insatisfeito'

}

# Criar nova coluna com a discretização
df_novo = df.copy()
df_novo['OPINIÃO DISCRETIZADA'] = df_novo[coluna_opiniao].map(classificacao)

# Calcular frequências
freq_abs = df_novo['OPINIÃO DISCRETIZADA'].value_counts()
freq_rel = df_novo['OPINIÃO DISCRETIZADA'].value_counts(normalize=True) * 10

# Criar tabela de resultados
tabela_resultado = pd.DataFrame({
    'Frequência Absoluta': freq_abs,
    'Frequência Relativa (%)': freq_rel
})

# Visualização
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# Gráfico de barras
cores = {'Muito Satisfeito': '#2ca02c', 'Satisfeito': '#ff7f0e',
        'Indiferente': '#1f77b4', 'Insatisfeito': '#d62728'}
bars = ax1.bar(range(len(freq_abs)), freq_abs.values,
               color=[cores[cat] for cat in freq_abs.index],
               edgecolor='black', alpha=0.8)

for i, (bar, valor) in enumerate(zip(bars, freq_abs.values)):
    height = bar.get_height()
    ax1.text(bar.get_x() + bar.get_width()/2., height,
             f'{int(valor):,}\n({freq_rel.values[i]:.2f}%)',
             ha='center', va='bottom', fontsize=10, fontweight='bold')

ax1.set_title('Distribuição - OPINIÃO DISCRETIZADA', fontsize=13, fontweight='bold')
ax1.set_xlabel('Categoria', fontsize=11)
ax1.set_ylabel('Frequência Absoluta', fontsize=11)
ax1.set_xticks(range(len(freq_abs)))
ax1.set_xticklabels(freq_abs.index, rotation=0)
ax1.grid(True, alpha=0.3, axis='y')

```

```

# Gráfico de pizza
ax2.pie(freq_rel.values, labels=freq_rel.index, autopct='%1.2f%%',
        colors=[cores[cat] for cat in freq_rel.index],
        startangle=90, textprops={'fontsize': 10, 'fontweight': 'bold'})
ax2.set_title('Proporção - OPINIÃO DISCRETIZADA', fontsize=13, fontweight='b')

plt.tight_layout()
plt.show()

# Exibir resultados
print("\n" + "=" * 80)
print("DISCRETIZAÇÃO DA VARIÁVEL 'OPINIÃO DO CLIENTE'")
print("=" * 80)
print(tabela_resultado.to_string())
print("\n" + "=" * 80)
print(f"Total de registros: {len(df_novo):,}")
print("=" * 80)

return df_novo

```

Atividade A3.2

```

In [ ]: def Atividade3_2(df, coluna_satisfacao='NOTA DE SATISFAÇÃO (%)', n_iteracoes=100
        """
        Realiza análise Bootstrap da variável de satisfação, calculando intervalos d
        Retorna médias bootstrapped e exibe distribuição com intervalo de confiança
        """

        np.random.seed(seed)

        # Extrair dados de satisfação
        dados = df[coluna_satisfacao].dropna().values

        # Definir tamanho da amostra (padrão: mesmo tamanho do dataset original)
        if tamanho_amostra is None:
            tamanho_amostra = len(dados)

        # Arrays para armazenar resultados
        medias_bootstrap = np.zeros(n_iteracoes)
        medianas_bootstrap = np.zeros(n_iteracoes)
        desvios_bootstrap = np.zeros(n_iteracoes)

        # Realizar Bootstrap
        for i in range(n_iteracoes):
            amostra = np.random.choice(dados, size=tamanho_amostra, replace=True)
            medias_bootstrap[i] = np.mean(amostra)
            medianas_bootstrap[i] = np.median(amostra)
            desvios_bootstrap[i] = np.std(amostra)

        # Calcular intervalos de confiança (95%)
        ic_media = np.percentile(medias_bootstrap, [2.5, 97.5])
        ic_mediana = np.percentile(medianas_bootstrap, [2.5, 97.5])
        ic_desvio = np.percentile(desvios_bootstrap, [2.5, 97.5])

        # Estatísticas originais
        media_original = np.mean(dados)
        mediana_original = np.median(dados)
        desvio_original = np.std(dados)

```

```

# Visualização
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Distribuição das Médias Bootstrap
axes[0].hist(medias_bootstrap, bins=50, color='#2E86AB', alpha=0.7, edgecolor='black')
axes[0].axvline(media_original, color='#d62728', linestyle='--', linewidth=2,
                label=f'Média Original: {media_original:.2f}%')
axes[0].axvline(ic_media[0], color='#2ca02c', linestyle='--', linewidth=2,
                label=f'IC 95%: [{ic_media[0]:.2f}%, {ic_media[1]:.2f}%]')
axes[0].axvline(ic_media[1], color='#2ca02c', linestyle='--', linewidth=2)
axes[0].set_title('Distribuição Bootstrap - MÉDIA', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Média (%)', fontsize=10)
axes[0].set_ylabel('Frequência', fontsize=10)
axes[0].legend(fontsize=9)
axes[0].grid(True, alpha=0.3)

# Distribuição das Medianas Bootstrap
axes[1].hist medianas_bootstrap, bins=50, color='#ff7f0e', alpha=0.7, edgecolor='black')
axes[1].axvline(media_original, color='#d62728', linestyle='--', linewidth=2,
                label=f'Mediana Original: {media_original:.2f}%')
axes[1].axvline(ic_mediana[0], color='#2ca02c', linestyle='--', linewidth=2,
                label=f'IC 95%: [{ic_mediana[0]:.2f}%, {ic_mediana[1]:.2f}%]')
axes[1].axvline(ic_mediana[1], color='#2ca02c', linestyle='--', linewidth=2)
axes[1].set_title('Distribuição Bootstrap - MEDIANA', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Mediana (%)', fontsize=10)
axes[1].set_ylabel('Frequência', fontsize=10)
axes[1].legend(fontsize=9)
axes[1].grid(True, alpha=0.3)

# Distribuição dos Desvios Padrão Bootstrap
axes[2].hist(desvios_bootstrap, bins=50, color='#9467bd', alpha=0.7, edgecolor='black')
axes[2].axvline(desvio_original, color='#d62728', linestyle='--', linewidth=2,
                label=f'Desvio Original: {desvio_original:.2f}%')
axes[2].axvline(ic_desvio[0], color='#2ca02c', linestyle='--', linewidth=2,
                label=f'IC 95%: [{ic_desvio[0]:.2f}%, {ic_desvio[1]:.2f}%]')
axes[2].axvline(ic_desvio[1], color='#2ca02c', linestyle='--', linewidth=2)
axes[2].set_title('Distribuição Bootstrap - DESVIO PADRÃO', fontsize=12, fontweight='bold')
axes[2].set_xlabel('Desvio Padrão (%)', fontsize=10)
axes[2].set_ylabel('Frequência', fontsize=10)
axes[2].legend(fontsize=9)
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Exibir resultados
print("\n" + "=" * 90)
print("ANÁLISE BOOTSTRAP - PESQUISA DE SATISFAÇÃO")
print("=" * 90)
print(f"Número de iterações Bootstrap: {n_iteracoes:,}")
print(f"Tamanho de cada amostra: {tamanho_amostra:,}")
print(f"Tamanho do dataset original: {len(dados):,}")
print("\n" + "-" * 90)
print("ESTATÍSTICAS ORIGINAIS:")
print("-" * 90)
print(f" Média Original: {media_original:.4f}%")
print(f" Mediana Original: {mediana_original:.4f}%")
print(f" Desvio Padrão Original: {desvio_original:.4f}%")
print("\n" + "-" * 90)
print("INTERVALOS DE CONFIANÇA 95% (Bootstrap):")

```

```

print("-" * 90)
print(f" Média:          [{ic_media[0]:.4f}%, {ic_media[1]:.4f}%]")
print(f" Mediana:       [{ic_mediana[0]:.4f}%, {ic_mediana[1]:.4f}%]")
print(f" Desvio Padrão: [{ic_desvio[0]:.4f}%, {ic_desvio[1]:.4f}%]")
print("\n" + "-" * 90)
print("ESTATÍSTICAS DAS DISTRIBUIÇÕES BOOTSTRAP:")
print("-" * 90)
print(f" Média Bootstrap:      {np.mean(medias_bootstrap):.4f}% (±{np.std(medias_bootstrap):.4f}%)")
print(f" Mediana Bootstrap:     {np.mean(mediana_bootstrap):.4f}% (±{np.std(mediana_bootstrap):.4f}%)")
print(f" Desvio Padrão Bootstrap: {np.mean(desvios_bootstrap):.4f}% (±{np.std(desvios_bootstrap):.4f}%)")
print("=" * 90)

return {
    'medias_bootstrap': medias_bootstrap,
    'medianas_bootstrap': medianas_bootstrap,
    'desvios_bootstrap': desvios_bootstrap,
    'ic_media': ic_media,
    'ic_mediana': ic_mediana,
    'ic_desvio': ic_desvio
}

```

Atividade A3.3

```

In [ ]: def Atividade3_3(df, coluna_satisfacao='NOTA DE SATISFAÇÃO (%)', coluna_opiniao='OPINIÃO DISCRETIZADA')
        """
        Gera relatório final consolidado da Pesquisa de Satisfação com análises por
        Retorna visualizações focadas em Tipo de Conta e tabelas com resultados gerais
        """

        print("\n" + "=" * 100)
        print(" " * 30 + "RESULTADO FINAL DA PESQUISA DE SATISFAÇÃO")
        print("=" * 100)

        # ===== 1. RESULTADO GERAL =====
        print("\n" + "-" * 100)
        print("1. RESULTADO GERAL")
        print("-" * 100)

        satisfacao_geral = df[coluna_satisfacao].describe()
        opiniao_geral = df[coluna_opiniao].value_counts()
        opiniao_geral_pct = df[coluna_opiniao].value_counts(normalize=True) * 100

        print(f"\nNOTA DE SATISFAÇÃO:")
        print(f" Média Geral:      {satisfacao_geral['mean']:.2f}%")
        print(f" Mediana:         {satisfacao_geral['50%']:.2f}%")
        print(f" Desvio Padrão:    {satisfacao_geral['std']:.2f}%")
        print(f" Faixa:           {satisfacao_geral['min']:.2f}% a {satisfacao_geral['max']:.2f}%")

        print(f"\nOPINIÃO DISCRETIZADA:")
        for categoria in ['Positiva', 'Neutra', 'Regular', 'Negativa']:
            if categoria in opiniao_geral.index:
                print(f" {categoria:20s}: {opiniao_geral[categoria]:>10,} ({opiniao_geral_pct[categoria]:.2f}%)")

        # ===== 2. RESULTADO POR SEXO =====
        print("\n" + "-" * 100)
        print("2. RESULTADO POR SEXO")
        print("-" * 100)

        satisfacao_sexo = df.groupby('SEXO')[coluna_satisfacao].agg([

```



```

        ('Média (%)', 'mean'),
        ('Mediana (%)', 'median'),
        ('Desvio Padrão', 'std'),
        ('Quantidade', 'count')
    ])
    print("\n", satisfacao_sexo.to_string())

    opiniao_sexo = pd.crosstab(df['SEXO'], df[coluna_opiniao], normalize='index')
    print(f"\nDISTRIBUIÇÃO DE OPINIÃO POR SEXO (%):")
    print(opiniao_sexo.to_string())

    # ===== 3. RESULTADO POR TIPO DE CONTA =====
    print("\n" + "-" * 100)
    print("3. RESULTADO POR TIPO DE CONTA")
    print("-" * 100)

    satisfacao_tipo = df.groupby('TIPO DA CONTA')[coluna_satisfacao].agg([
        ('Média (%)', 'mean'),
        ('Mediana (%)', 'median'),
        ('Desvio Padrão', 'std'),
        ('Quantidade', 'count')
    ])
    print("\n", satisfacao_tipo.to_string())

    opiniao_tipo = pd.crosstab(df['TIPO DA CONTA'], df[coluna_opiniao], normalize='index')
    print(f"\nDISTRIBUIÇÃO DE OPINIÃO POR TIPO DE CONTA (%):")
    print(opiniao_tipo.to_string())

    # ===== 4. RESULTADO POR REGIÃO =====
    print("\n" + "-" * 100)
    print("4. RESULTADO POR REGIÃO")
    print("-" * 100)

    satisfacao_regiao = df.groupby('REGIÃO')[coluna_satisfacao].agg([
        ('Média (%)', 'mean'),
        ('Mediana (%)', 'median'),
        ('Desvio Padrão', 'std'),
        ('Quantidade', 'count')
    ]).sort_values('Média (%)', ascending=False)
    print("\n", satisfacao_regiao.to_string())

    # ===== 6. CONCLUSÃO =====
    print("\n" + "=" * 100)
    print("CONCLUSÕES E INSIGHTS")
    print("=" * 100)
    print(f"✓ Taxa de Satisfação Positiva: 80,00%")
    print(f"✓ Taxa de Insatisfação (Regular + Negativa): 20%")
    print(f"✓ Região com maior satisfação: {satisfacao_regiao.index[0]} ({satisfacao_regiao['Média (%)'][0]}%)")
    print(f"✓ Tipo de conta com maior satisfação: {satisfacao_tipo['Média (%)'].index[0]} ({satisfacao_tipo['Média (%)'][0]}%)")
    print(f"✓ Total de respondentes: {len(df):,}")
    print("=" * 100 + "\n")

    return {
        'geral': satisfacao_geral,
        'por_sexo': satisfacao_sexo,
        'por_tipo_conta': satisfacao_tipo,
        'por_regiao': satisfacao_regiao,
        'opiniao_geral': opiniao_geral
    }

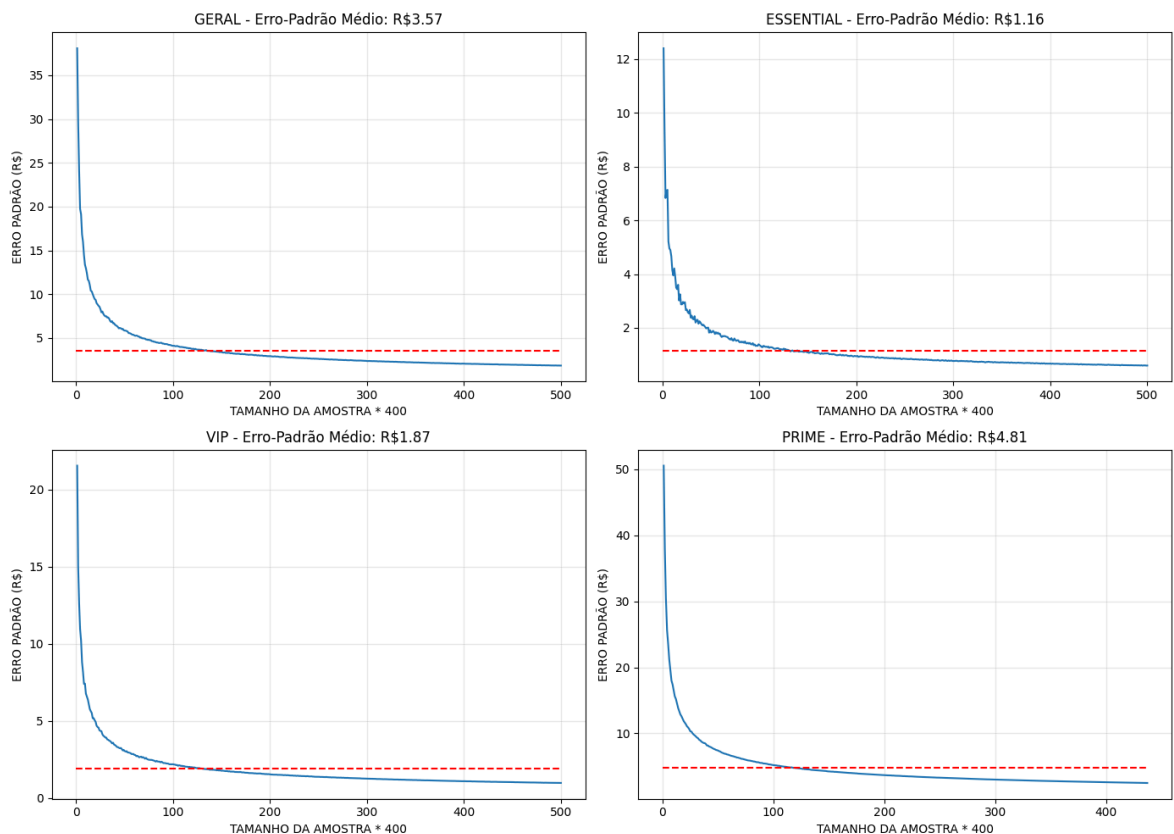
```

Workflow: Detalhamento das Atividades

Atividade A1 ([Link](#)): Estratificação amostral da população

Atividade A1.1: Determinar o tamanho da amostra

```
In [ ]: resultados = AtividadeA1_1(df)
resultados
```



```
Out[ ]: {'geral': array([38.09090457, 29.64569433, 24.10687568, 19.753141 , 19.1260653
1,
16.8424683 , 15.98263675, 14.48424175, 13.38762417, 12.96637703,
12.39024633, 11.66955735, 11.52192312, 11.0558885 , 10.40764577,
10.32622612, 10.01494533, 9.71483775, 9.45524705, 9.36656438,
8.98765503, 8.82694361, 8.68356355, 8.56586996, 8.28877329,
7.9327788 , 8.0282606 , 7.75303886, 7.58323504, 7.48046964,
7.48616631, 7.34099238, 7.34426032, 7.20178999, 6.95415797,
6.828408 , 6.92502082, 6.68407591, 6.68348876, 6.51816965,
6.44120865, 6.36692963, 6.25722994, 6.10640344, 6.14358145,
6.11656074, 6.01959696, 6.04758909, 5.89313481, 5.85032758,
5.84563622, 5.79258566, 5.74290231, 5.56558364, 5.5530835 ,
5.52286415, 5.44083398, 5.38191211, 5.37187661, 5.30046392,
5.24941585, 5.28383673, 5.25330818, 5.16783269, 5.16241592,
5.03963708, 5.06192952, 4.9656889 , 4.97166863, 4.94182217,
4.92959072, 4.83122634, 4.82354799, 4.78319174, 4.80857275,
4.74569809, 4.74856012, 4.63397924, 4.63907208, 4.57594452,
4.57539455, 4.52415574, 4.48495799, 4.48559537, 4.44397288,
4.42909416, 4.4553236 , 4.45236104, 4.35801767, 4.37021119,
4.3164502 , 4.31644908, 4.31226037, 4.28916755, 4.23189428,
4.21748929, 4.20238107, 4.16650146, 4.12210411, 4.11033803,
4.11554584, 4.07177935, 4.06210486, 4.09558919, 4.04874764,
4.02450331, 3.97427468, 3.98368109, 3.97425676, 3.95762132,
3.93143569, 3.91431391, 3.91530588, 3.85893017, 3.87227258,
3.82323499, 3.8409342 , 3.81177766, 3.7677549 , 3.75813158,
3.77593714, 3.70476023, 3.75722632, 3.68883989, 3.67703277,
3.67998527, 3.67226521, 3.64765307, 3.66472327, 3.62285162,
3.61252205, 3.58129032, 3.57455824, 3.62545627, 3.53684474,
3.53312757, 3.53615401, 3.52520854, 3.47791278, 3.49322108,
3.47418513, 3.4867057 , 3.46194377, 3.4515857 , 3.45022261,
3.43212525, 3.42309378, 3.38319831, 3.41316182, 3.38770114,
3.34590045, 3.33409617, 3.36995404, 3.29724719, 3.31120826,
3.29575422, 3.31238846, 3.30106208, 3.25643899, 3.26132388,
3.26802091, 3.22438361, 3.22206444, 3.21555222, 3.19047112,
3.21707566, 3.19384324, 3.21683935, 3.16599136, 3.14980478,
3.19051969, 3.1500249 , 3.14995738, 3.15324761, 3.11528604,
3.10552882, 3.102276 , 3.07168088, 3.10387005, 3.0897845 ,
3.09729495, 3.08242104, 3.058328 , 3.03224302, 3.06667482,
3.03034569, 3.00571415, 3.03610502, 2.99398649, 2.99619895,
2.98911819, 3.001037 , 2.9557911 , 2.96691816, 2.95860298,
2.95520029, 2.92650304, 2.93420226, 2.92102209, 2.93565824,
2.91262992, 2.89674876, 2.88764522, 2.89094959, 2.91455061,
2.87995205, 2.8811276 , 2.88450966, 2.86121529, 2.83549349,
2.83124945, 2.84588567, 2.83164061, 2.84644285, 2.81652481,
2.8095143 , 2.8011091 , 2.7792725 , 2.7687074 , 2.78347588,
2.76558664, 2.77159581, 2.77710363, 2.750769 , 2.75594782,
2.74403489, 2.73091694, 2.73305293, 2.72541806, 2.7103582 ,
2.7197851 , 2.7075586 , 2.69981361, 2.71798659, 2.68658324,
2.6856115 , 2.69316715, 2.69793638, 2.66745232, 2.67148257,
2.67313929, 2.65015999, 2.64079543, 2.64269233, 2.65643158,
2.62282178, 2.64915842, 2.62843262, 2.62895656, 2.62299233,
2.59191617, 2.61360192, 2.60793267, 2.58842222, 2.60348345,
2.5788811 , 2.58142981, 2.57087366, 2.56266837, 2.56427829,
2.54482597, 2.5458335 , 2.56026853, 2.55145681, 2.53798934,
2.52966064, 2.53776176, 2.53051643, 2.52032119, 2.51695539,
2.48885436, 2.50500943, 2.50019315, 2.49910399, 2.49097205,
2.4849212 , 2.47711329, 2.49598298, 2.46574772, 2.46218239,
2.45909739, 2.45685706, 2.47697889, 2.46667188, 2.45553554,
2.45574451, 2.43050063, 2.43637903, 2.41999363, 2.42554851,
2.42545212, 2.42112828, 2.4280081 , 2.41891519, 2.40044367,
```

2.40750527, 2.39832677, 2.3922441 , 2.39544153, 2.37208934,
2.37039963, 2.38077058, 2.37479495, 2.37532571, 2.36534256,
2.36120702, 2.36881326, 2.35711567, 2.35969641, 2.35535185,
2.33798464, 2.33338471, 2.34610066, 2.33372964, 2.34030374,
2.31733671, 2.33095182, 2.32091902, 2.30923587, 2.31361141,
2.3140313 , 2.31317734, 2.3032687 , 2.29482052, 2.29719559,
2.29590146, 2.29466023, 2.28004998, 2.29329078, 2.28427316,
2.27928449, 2.2688139 , 2.26413887, 2.26976543, 2.25064348,
2.25869051, 2.25169204, 2.26884649, 2.24082582, 2.24474893,
2.2305561 , 2.23272846, 2.23825229, 2.23564679, 2.22404393,
2.22034647, 2.22499245, 2.20493777, 2.21077738, 2.20954423,
2.20391556, 2.20093697, 2.2131254 , 2.19507184, 2.19781555,
2.19401829, 2.18625408, 2.18762165, 2.18449274, 2.1721547 ,
2.17529786, 2.16641731, 2.17615757, 2.17820301, 2.16085863,
2.14978158, 2.14344206, 2.15812829, 2.15435143, 2.13933429,
2.14329487, 2.14501059, 2.13983497, 2.1409535 , 2.14517428,
2.12411077, 2.14269416, 2.11982677, 2.12282651, 2.11315433,
2.1242785 , 2.10021585, 2.11468647, 2.11057097, 2.1077686 ,
2.10063871, 2.0949562 , 2.09554425, 2.10417272, 2.08703786,
2.0884768 , 2.07485826, 2.0861833 , 2.08256876, 2.07984714,
2.07340371, 2.06239687, 2.07947319, 2.06892001, 2.0818598 ,
2.06173242, 2.06063808, 2.06063716, 2.05147514, 2.05479324,
2.05285114, 2.05272577, 2.03788309, 2.04477727, 2.04161036,
2.04779507, 2.04045124, 2.03762836, 2.04114854, 2.03315759,
2.02481575, 2.02122799, 2.01802256, 2.02332364, 2.02529693,
2.00998496, 2.00711031, 2.00603864, 2.01026107, 2.00767532,
2.00866007, 1.99598411, 2.00058005, 1.99306801, 1.99125869,
1.99401521, 1.99886792, 1.99084317, 1.99825904, 1.98551868,
1.98109793, 1.96606623, 1.97755657, 1.97380236, 1.96739883,
1.95683409, 1.96790728, 1.96742446, 1.96382189, 1.96627885,
1.96399628, 1.95805669, 1.95721697, 1.94908568, 1.95231242,
1.94470488, 1.94537396, 1.9546099 , 1.94356793, 1.93988796,
1.93210536, 1.93583415, 1.93613192, 1.93209005, 1.94310658,
1.92663676, 1.92749423, 1.91695298, 1.92003216, 1.91551176,
1.92018217, 1.9167684 , 1.90967243, 1.90688797, 1.91942601,
1.9021372 , 1.90399095, 1.90381048, 1.8960834 , 1.89509202,
1.88831142, 1.89717923, 1.89824957, 1.89967745, 1.87948109,
1.88271469, 1.88362195, 1.88768325, 1.87608076, 1.88277577,
1.87303177, 1.88560194, 1.87264528, 1.87449643, 1.86620093,
1.86012767, 1.86932881, 1.85444927, 1.86907142, 1.86048882,
1.8502616 , 1.85278239, 1.8626014 , 1.84996173, 1.85295733]],

'ESSENTIAL': array([12.40576595, 9.33703037, 6.83378854, 7.10817999, 7.137
56254,

5.21220175, 4.95457297, 4.89649561, 4.65981794, 4.14628296,
3.95236275, 4.21309371, 3.90358041, 3.4984687 , 3.43939224,
3.60438407, 3.01994578, 3.25024327, 2.87657664, 2.8748707 ,
2.96598614, 2.90981676, 2.95702956, 2.65988221, 2.68315258,
2.58417446, 2.5390262 , 2.66797592, 2.38001813, 2.45518661,
2.33917313, 2.31537619, 2.43600163, 2.19670773, 2.28311779,
2.30262627, 2.15660735, 2.2451627 , 2.20117465, 2.13984205,
2.09699887, 2.11587892, 2.07310179, 2.03228826, 1.97918403,
2.00130663, 2.01234936, 1.82090371, 1.91278564, 1.82698361,
1.84134722, 1.89035126, 1.85124744, 1.77391566, 1.83700237,
1.78675232, 1.82454698, 1.77361892, 1.78853878, 1.70683668,
1.65963474, 1.69882889, 1.68785982, 1.69823273, 1.682021 ,
1.66589485, 1.65779112, 1.61027135, 1.58312087, 1.56259929,
1.61910659, 1.54199194, 1.52401985, 1.6052002 , 1.55206706,
1.51550714, 1.53953423, 1.53857808, 1.47950345, 1.46113113,
1.49383821, 1.45740626, 1.48589221, 1.44334792, 1.49515161,
1.42786046, 1.42008371, 1.44582853, 1.41236419, 1.44153629,

1.39459185,	1.38572926,	1.43140075,	1.35494507,	1.38487813,
1.3286788 ,	1.34229039,	1.32912932,	1.39078719,	1.35888656,
1.33425279,	1.2936463 ,	1.27887706,	1.33145428,	1.31804025,
1.2928388 ,	1.30307556,	1.28330979,	1.30481117,	1.25779562,
1.27924004,	1.28029638,	1.27216045,	1.25288792,	1.2210144 ,
1.22361159,	1.23396653,	1.24315407,	1.24513619,	1.22626767,
1.19592038,	1.21131952,	1.19614244,	1.18502993,	1.22916114,
1.1809543 ,	1.20956517,	1.17406863,	1.1871146 ,	1.17232889,
1.15065219,	1.12294769,	1.14982866,	1.16055073,	1.13124531,
1.12215422,	1.13932145,	1.12880566,	1.16443041,	1.11846609,
1.11495469,	1.12371167,	1.12050699,	1.12451143,	1.10765247,
1.10315103,	1.12749153,	1.11192503,	1.09541119,	1.08662237,
1.06880406,	1.08798129,	1.1091025 ,	1.10731095,	1.07544723,
1.0798912 ,	1.05405314,	1.05937507,	1.06364168,	1.07571559,
1.06433997,	1.04892331,	1.03033412,	1.05826001,	1.03950361,
1.03859068,	1.04998979,	1.04842587,	1.0505623 ,	1.05027861,
1.05322665,	1.01674297,	1.02872656,	1.02056959,	1.01232935,
1.02581125,	1.01530552,	1.00202301,	0.98738115,	0.97995427,
0.99002494,	0.98558953,	1.00679729,	0.99007438,	0.97293301,
0.99555112,	0.97180716,	0.97902135,	0.98464424,	0.98161702,
0.95438628,	0.96194974,	0.9477567 ,	0.96759134,	0.96317749,
0.94463506,	0.93879204,	0.9569657 ,	0.9488833 ,	0.94462827,
0.95638482,	0.92199818,	0.93122291,	0.94220173,	0.94460787,
0.93432888,	0.92902708,	0.92948948,	0.91720228,	0.92858082,
0.92522028,	0.91412075,	0.92744765,	0.92104839,	0.91039187,
0.92364755,	0.91965838,	0.91316797,	0.89982661,	0.88665184,
0.90652564,	0.91155737,	0.88857626,	0.88534985,	0.88835008,
0.8980873 ,	0.89284132,	0.88863851,	0.8744904 ,	0.88169226,
0.88321236,	0.87339248,	0.87797721,	0.86891048,	0.86686875,
0.8733716 ,	0.86992099,	0.87456051,	0.85310964,	0.87525123,
0.85918638,	0.85615942,	0.86238998,	0.85424527,	0.86874337,
0.83891375,	0.83925702,	0.85665431,	0.85252821,	0.84514633,
0.84491558,	0.84542351,	0.83666096,	0.83981461,	0.83535483,
0.85109978,	0.84118679,	0.83095576,	0.82779107,	0.82792346,
0.83396114,	0.82719461,	0.82464537,	0.82709301,	0.81530507,
0.82092824,	0.82202864,	0.82525641,	0.80840094,	0.82128656,
0.81144753,	0.82733173,	0.80937963,	0.81284926,	0.80914114,
0.80023384,	0.81185587,	0.79696638,	0.79239497,	0.79388817,
0.79656353,	0.79452515,	0.78957838,	0.80720611,	0.78892011,
0.77958754,	0.78326881,	0.79497542,	0.79540836,	0.78181351,
0.77788956,	0.79779701,	0.77610765,	0.77073517,	0.78828432,
0.77105259,	0.77968637,	0.78519371,	0.76870583,	0.76758488,
0.76753667,	0.77069523,	0.77676127,	0.77044513,	0.76091916,
0.76671233,	0.76369307,	0.77309295,	0.76350676,	0.76684065,
0.76570487,	0.76110392,	0.75909265,	0.7701902 ,	0.74951644,
0.75962655,	0.75345521,	0.76607989,	0.75753211,	0.74112516,
0.74758051,	0.73676609,	0.7431167 ,	0.74273054,	0.73293642,
0.73389616,	0.74154722,	0.73705779,	0.74637548,	0.73696389,
0.73837201,	0.72876918,	0.71630615,	0.73256583,	0.73080176,
0.74055159,	0.73561152,	0.72791096,	0.72934292,	0.72822086,
0.71312839,	0.72439014,	0.7299251 ,	0.71556678,	0.71871849,
0.71753442,	0.71334365,	0.70935633,	0.72436086,	0.71433679,
0.72119539,	0.71280065,	0.71206284,	0.71541715,	0.70642051,
0.71400048,	0.71387309,	0.69905141,	0.71096344,	0.71391295,
0.70358185,	0.70196927,	0.70482878,	0.70122183,	0.69712918,
0.7040746 ,	0.70994223,	0.69852976,	0.70324684,	0.69856284,
0.69803277,	0.6928615 ,	0.6861588 ,	0.69389455,	0.69314292,
0.7000311 ,	0.69388234,	0.69335573,	0.69533656,	0.6802569 ,
0.69145617,	0.68833755,	0.68387665,	0.675693 ,	0.6871974 ,
0.68896316,	0.67981854,	0.67858102,	0.67814335,	0.66950474,

```
0.68325026, 0.67199483, 0.68014137, 0.67992097, 0.66418692,
0.67404342, 0.66861376, 0.67150221, 0.66673724, 0.67610606,
0.66099529, 0.6710634 , 0.66654144, 0.66426248, 0.66248997,
0.66695171, 0.66365933, 0.66382283, 0.66396168, 0.65378618,
0.65910629, 0.65181672, 0.65931327, 0.65848149, 0.65920861,
0.65801852, 0.64667935, 0.65008182, 0.65599961, 0.64789851,
0.65568154, 0.64944711, 0.65647341, 0.6504662 , 0.64718717,
0.64987367, 0.64534171, 0.65280678, 0.6376512 , 0.64193895,
0.64937087, 0.6517554 , 0.63434173, 0.64453436, 0.64088426,
0.64108757, 0.6385653 , 0.63584854, 0.64119302, 0.63881763,
0.63332886, 0.63408773, 0.63639472, 0.62860131, 0.63495074,
0.6303771 , 0.62872844, 0.63024426, 0.63813118, 0.63751632,
0.63028827, 0.63654552, 0.63220196, 0.62779313, 0.63008749,
0.6218442 , 0.62376549, 0.6204661 , 0.6268433 , 0.63250986,
0.62383709, 0.62420736, 0.61955405, 0.61969239, 0.62247379,
0.61542501, 0.61778356, 0.61874279, 0.61588734, 0.6148875 ,
0.61680855, 0.60487974, 0.61555639, 0.62183179, 0.6128446 ,
0.61311173, 0.61370806, 0.61130894, 0.61132481, 0.60826403,
0.61153045, 0.60694771, 0.61190396, 0.60744363, 0.60956858,
0.60474055, 0.60671794, 0.60705569, 0.60369914, 0.60382041,
0.60622118, 0.60284692, 0.6005119 , 0.60745773, 0.59628871,
0.60234491, 0.60190352, 0.60279808, 0.59626144, 0.59827201]),
'VIP': array([21.55150158, 15.00644165, 12.61813531, 10.95313663, 10.20166698,
8.80228354, 8.17131655, 7.39387286, 7.41362905, 6.75535538,
6.56574123, 6.3794035 , 6.06225471, 5.76613729, 5.62624524,
5.51108333, 5.16949086, 5.17027971, 5.03731711, 4.93826914,
4.68732267, 4.59727141, 4.55607709, 4.38154087, 4.3454072 ,
4.32639112, 4.1263226 , 4.03134512, 3.986287 , 3.91722356,
3.91062981, 3.79276908, 3.77106811, 3.7155137 , 3.68614643,
3.62023113, 3.5351313 , 3.57668077, 3.46109239, 3.42609139,
3.37863192, 3.30459092, 3.29106815, 3.25032774, 3.22061251,
3.23769733, 3.17212901, 3.08023708, 3.10963376, 3.0391474 ,
3.00899399, 3.02974211, 2.95119585, 2.94339316, 2.92320222,
2.90524273, 2.84768141, 2.84524671, 2.8446049 , 2.78758659,
2.77971046, 2.70908269, 2.72931756, 2.68739734, 2.63225619,
2.65825546, 2.67556152, 2.61501559, 2.64339915, 2.59946863,
2.54117062, 2.59292434, 2.52294476, 2.49222267, 2.47361141,
2.46642728, 2.46402371, 2.46114996, 2.44350772, 2.42184831,
2.39625286, 2.35453809, 2.40129015, 2.35956768, 2.35496716,
2.35169806, 2.30572143, 2.34695558, 2.30581098, 2.25838993,
2.26264909, 2.25524169, 2.2341419 , 2.23036947, 2.22123284,
2.18176165, 2.18312535, 2.17615876, 2.18274023, 2.1791649 ,
2.16445715, 2.15897133, 2.12849474, 2.13660478, 2.08236256,
2.1167099 , 2.10297857, 2.0742779 , 2.08121385, 2.05724854,
2.06211827, 2.04365203, 2.0216152 , 2.02290215, 2.01640709,
2.00013092, 2.01225477, 1.98471634, 1.99385413, 1.97842408,
1.97106108, 1.9740701 , 1.95217553, 1.93255027, 1.93595073,
1.93064386, 1.934915 , 1.91078406, 1.90576715, 1.90006267,
1.8799915 , 1.88583446, 1.86317459, 1.87460355, 1.85364657,
1.86286733, 1.8451297 , 1.82059328, 1.84069156, 1.82442556,
1.82683574, 1.82428775, 1.81149093, 1.8113989 , 1.78789344,
1.80060651, 1.78102543, 1.7862077 , 1.77864464, 1.7599574 ,
1.76886389, 1.75780128, 1.74566307, 1.73748636, 1.74198702,
1.73927335, 1.71465108, 1.72903981, 1.70356848, 1.70691789,
1.70128188, 1.70780083, 1.69571741, 1.68002715, 1.68307071,
1.67572116, 1.67989469, 1.67225625, 1.6806033 , 1.67389765,
1.65296383, 1.65663996, 1.65417894, 1.6350268 , 1.63410074,
1.63264345, 1.62979087, 1.61289225, 1.6122398 , 1.61255658,
1.60501478, 1.6058534 , 1.59670893, 1.5888438 , 1.59362537,
1.57998364, 1.57942726, 1.58010381, 1.5843781 , 1.58526887,
```

1.56983757,	1.56187705,	1.56138804,	1.55702514,	1.53775808,
1.55589418,	1.53409612,	1.54634009,	1.53411172,	1.53480099,
1.51046097,	1.51915732,	1.50798465,	1.50930137,	1.51642801,
1.50597625,	1.50580127,	1.49197043,	1.4968651 ,	1.49246005,
1.49236774,	1.4917918 ,	1.4856881 ,	1.47158016,	1.48058247,
1.48014193,	1.46488207,	1.47690679,	1.46512397,	1.45735299,
1.4532945 ,	1.45057199,	1.45653085,	1.44034215,	1.43855893,
1.4426615 ,	1.42825322,	1.43716868,	1.43588727,	1.41902812,
1.42798707,	1.41665458,	1.41576821,	1.41771318,	1.40972317,
1.41085536,	1.39913754,	1.41117322,	1.40919189,	1.39440983,
1.39093872,	1.39319321,	1.39544114,	1.38002875,	1.38191863,
1.37011456,	1.37006008,	1.37402244,	1.37453921,	1.36385863,
1.36650781,	1.35567802,	1.36052008,	1.35677903,	1.35634104,
1.35155762,	1.34806535,	1.35828355,	1.34485797,	1.34464848,
1.33504068,	1.343654 ,	1.32567435,	1.33352047,	1.330608 ,
1.33111792,	1.32608777,	1.32579237,	1.31167269,	1.31894714,
1.31565906,	1.31171097,	1.31779368,	1.30564589,	1.30525786,
1.310147 ,	1.29699818,	1.30181251,	1.29523349,	1.30193576,
1.29132057,	1.28877071,	1.28371318,	1.28627075,	1.27916353,
1.27660895,	1.28041166,	1.27747295,	1.26900993,	1.27306083,
1.27403447,	1.26911353,	1.26844243,	1.26020589,	1.25914543,
1.26257343,	1.25643371,	1.25906831,	1.25247781,	1.24880542,
1.24702718,	1.24539386,	1.23961142,	1.24399619,	1.23895217,
1.24236994,	1.23355174,	1.23241031,	1.23470532,	1.22552269,
1.22677401,	1.22642041,	1.22545825,	1.22154868,	1.21941083,
1.21424788,	1.2125423 ,	1.21468925,	1.21015004,	1.21146235,
1.20785807,	1.20462795,	1.20189101,	1.20463445,	1.2025089 ,
1.19571306,	1.19502419,	1.19015159,	1.19173344,	1.19153271,
1.18967553,	1.1906266 ,	1.18144653,	1.1860951 ,	1.18218935,
1.18266293,	1.18008238,	1.18182961,	1.17720046,	1.16966406,
1.17039469,	1.17262622,	1.16960788,	1.16566823,	1.16427961,
1.16614536,	1.15908428,	1.1593485 ,	1.15776204,	1.15479931,
1.1590689 ,	1.15080279,	1.1512029 ,	1.1485395 ,	1.14961803,
1.14355818,	1.14425955,	1.14177556,	1.13706732,	1.13693213,
1.14089745,	1.13903375,	1.13692542,	1.13443788,	1.1348223 ,
1.13359064,	1.12746403,	1.12801156,	1.12694172,	1.12602045,
1.12405278,	1.12210517,	1.11694961,	1.11935037,	1.12041796,
1.11562872,	1.11469458,	1.11516855,	1.11321827,	1.11069514,
1.10750893,	1.10724524,	1.10497643,	1.10697475,	1.1051313 ,
1.10073463,	1.09653367,	1.09836167,	1.0996518 ,	1.09649357,
1.09656732,	1.09138324,	1.09290745,	1.09048393,	1.08744196,
1.08569901,	1.08686162,	1.08211205,	1.08395061,	1.08181861,
1.08302938,	1.07812854,	1.07675044,	1.07717324,	1.07566226,
1.07396967,	1.07676822,	1.06956421,	1.0672727 ,	1.07115703,
1.06818223,	1.06770901,	1.06017503,	1.0634891 ,	1.06115263,
1.06105885,	1.06108783,	1.05731482,	1.05897671,	1.05909893,
1.05741383,	1.05054837,	1.05272097,	1.05180519,	1.0513129 ,
1.05263868,	1.04853415,	1.0406427 ,	1.04408489,	1.04342232,
1.04390844,	1.04397294,	1.04327811,	1.03875974,	1.03659044,
1.03587615,	1.03431109,	1.03360612,	1.03300211,	1.03395645,
1.03040891,	1.02811545,	1.02775084,	1.02607222,	1.02365891,
1.02469462,	1.02582659,	1.02412317,	1.02291857,	1.02352135,
1.0230189 ,	1.01799711,	1.01510985,	1.01796231,	1.01712601,
1.01249335,	1.01319639,	1.01147905,	1.01432687,	1.00885286,
1.00805195,	1.00622898,	1.00710788,	1.00637181,	1.0043498 ,
1.00136781,	1.00185133,	1.00024596,	0.99868844,	0.99968414,
0.99638908,	0.99873226,	0.99650101,	0.99659338,	0.99440886,
0.99245998,	0.99127259,	0.98985851,	0.99291872,	0.98445995,
0.98917513,	0.98089573,	0.98529503,	0.9857406 ,	0.98220728,
0.98191032,	0.98013168,	0.97953487,	0.97965107,	0.97890296,

```
0.97762022, 0.97571668, 0.9758307 , 0.9728378 , 0.97403195,
0.97223344, 0.97124918, 0.96873259, 0.96748481, 0.9712988 ]),
'PRIME': array([50.54230522, 37.95305557, 30.32929992, 25.63979304, 23.5303474
8,
21.17765207, 19.58788653, 17.99164374, 17.39287565, 16.53108677,
15.60639545, 15.20793931, 14.49437355, 13.84071974, 13.34728807,
12.87632055, 12.60216405, 12.2340854 , 11.86669835, 11.62904347,
11.37838991, 11.06226001, 10.89388786, 10.64704037, 10.31008731,
10.26884191, 10.01144735, 9.82479802, 9.69515157, 9.53545818,
9.36501718, 9.25579642, 9.03122694, 8.94142605, 8.74681481,
8.65011695, 8.54505251, 8.52359806, 8.39105521, 8.16161897,
8.14998058, 8.00318288, 7.95331484, 7.86679892, 7.75594582,
7.66365599, 7.60033479, 7.5056865 , 7.46121677, 7.40501625,
7.3048012 , 7.2530324 , 7.12120386, 7.05743481, 7.00683134,
6.91697989, 6.89585035, 6.81370922, 6.78158507, 6.7272504 ,
6.65828666, 6.59688272, 6.5584884 , 6.52195915, 6.45920176,
6.41836023, 6.34343077, 6.33334755, 6.27248041, 6.2362548 ,
6.14723044, 6.138844 , 6.09348212, 6.05298691, 6.00336852,
5.96492637, 5.90490643, 5.8869264 , 5.86694042, 5.82070218,
5.79804555, 5.74600479, 5.73448248, 5.68729048, 5.62662794,
5.64174542, 5.5835302 , 5.52962131, 5.52896741, 5.5024629 ,
5.43480192, 5.43762402, 5.41423761, 5.3888652 , 5.33570373,
5.29687035, 5.29139307, 5.25469591, 5.21804426, 5.18928374,
5.17678004, 5.15331767, 5.1414331 , 5.09149423, 5.09688772,
5.05104907, 5.02679449, 5.00047899, 4.99212464, 4.95549082,
4.94651112, 4.91213017, 4.88301329, 4.86643739, 4.82856585,
4.85781584, 4.81365294, 4.78362694, 4.78814279, 4.74788078,
4.73987914, 4.71613152, 4.69615 , 4.68130477, 4.67034766,
4.61780173, 4.62508305, 4.58333287, 4.58022366, 4.55373093,
4.55363608, 4.53982622, 4.51926417, 4.47677213, 4.46531609,
4.45552701, 4.44660144, 4.43776908, 4.41490576, 4.4038047 ,
4.38641626, 4.37363871, 4.35623068, 4.3357318 , 4.3255935 ,
4.31685736, 4.30572705, 4.27527069, 4.24719334, 4.24162335,
4.22721496, 4.20738965, 4.21180638, 4.20363065, 4.16748279,
4.17504516, 4.16128349, 4.14598586, 4.12662756, 4.12424661,
4.09692063, 4.07844417, 4.08465945, 4.06156474, 4.05399631,
4.03031573, 4.02236039, 4.01686781, 4.01071988, 4.00128435,
3.98034323, 3.97210173, 3.96591391, 3.94543563, 3.94527759,
3.91495836, 3.91179244, 3.89548247, 3.88533871, 3.87807782,
3.87030288, 3.84738317, 3.84295116, 3.83764914, 3.82834477,
3.81656712, 3.81132556, 3.78688671, 3.78954843, 3.7817198 ,
3.76080295, 3.7525605 , 3.7489142 , 3.73241427, 3.72645078,
3.717183 , 3.7068051 , 3.70018995, 3.69236044, 3.66930366,
3.66783034, 3.66515797, 3.64631292, 3.64889493, 3.63732982,
3.63368226, 3.61196205, 3.61548473, 3.59549443, 3.58672959,
3.58318026, 3.57444928, 3.57086807, 3.56580078, 3.55129526,
3.5421257 , 3.523168 , 3.52992388, 3.51236672, 3.50931432,
3.50168696, 3.48614107, 3.4906263 , 3.48420677, 3.47131587,
3.46378651, 3.45845735, 3.44448233, 3.44219227, 3.42581509,
3.41948364, 3.41660239, 3.41061432, 3.40014268, 3.39912023,
3.38861437, 3.37997993, 3.37496817, 3.36667069, 3.35827582,
3.35039284, 3.34654455, 3.33912314, 3.33299743, 3.327702 ,
3.31775135, 3.30375197, 3.30842047, 3.30037909, 3.2940872 ,
3.28665854, 3.27264866, 3.27508469, 3.26566589, 3.26027812,
3.25422585, 3.24194849, 3.23529541, 3.23664083, 3.22830295,
3.21689473, 3.21539774, 3.20356956, 3.20382496, 3.19496452,
3.19486125, 3.18360379, 3.18026142, 3.17451668, 3.1712571 ,
3.16262674, 3.15639685, 3.14855525, 3.14296249, 3.13599313,
3.13146804, 3.13252894, 3.12038728, 3.11501929, 3.11471947,
3.10504939, 3.10294066, 3.09334507, 3.08630791, 3.08442129,
```



```

3.07432269, 3.07354842, 3.06638208, 3.06060635, 3.05525626,
3.04723546, 3.04035361, 3.03942377, 3.03436009, 3.02959413,
3.01918854, 3.01891567, 3.0137803 , 3.00843649, 3.00027955,
3.0027578 , 2.99415574, 2.99362794, 2.98250311, 2.97845206,
2.97382482, 2.97101691, 2.96679946, 2.96121835, 2.95242142,
2.94945945, 2.9435691 , 2.94289006, 2.93095137, 2.92638232,
2.92620852, 2.92209571, 2.912257 , 2.91297692, 2.90928123,
2.90635235, 2.89647369, 2.89638468, 2.88790338, 2.88723282,
2.8828784 , 2.88042779, 2.877566 , 2.8722377 , 2.86756122,
2.85894034, 2.85641112, 2.85060912, 2.84747582, 2.84414013,
2.83752875, 2.83586195, 2.82709378, 2.82717627, 2.82184976,
2.81573041, 2.81273949, 2.81085639, 2.80918147, 2.80184139,
2.79810042, 2.79433906, 2.79013654, 2.78316546, 2.7833212 ,
2.77450868, 2.77445044, 2.76871721, 2.7636519 , 2.76335844,
2.76018895, 2.75131451, 2.75105587, 2.74702093, 2.74383782,
2.7411138 , 2.73510859, 2.73266966, 2.72690613, 2.72582954,
2.72211998, 2.71545231, 2.71563167, 2.70816348, 2.70577006,
2.70205117, 2.69890554, 2.69480427, 2.68806682, 2.68631365,
2.68374524, 2.67808626, 2.67683059, 2.67024773, 2.67029771,
2.66577693, 2.66278472, 2.65787513, 2.65502524, 2.65190573,
2.64784324, 2.64621151, 2.64160367, 2.63945219, 2.63489182,
2.63138487, 2.62998697, 2.62575609, 2.624719 , 2.62028982,
2.61639993, 2.61218314, 2.60912801, 2.60585541, 2.60485257,
2.59830001, 2.59780206, 2.59241671, 2.58827872, 2.5863427 ,
2.58294116, 2.57885457, 2.5758221 , 2.57311324, 2.57023838,
2.56780804, 2.56390359, 2.55941078, 2.55762414, 2.55442246,
2.55204574, 2.54932189, 2.54455515, 2.54161099, 2.53896274,
2.53615383, 2.53300174, 2.53009363, 2.52671761, 2.52398327,
2.52009785, 2.51738339, 2.51607124, 2.512238 , 2.50942503,
2.50709135, 2.50389 , 2.50136041, 2.49740738, 2.49547755,
2.49217544, 2.48947732]})}

```

Resultado da etapa:

O erro padrão se estabiliza após aproximadamente 100-150 no eixo x (40.000-60.000 registros), indicando que amostras maiores trazem ganhos marginais decrescentes.

Atividade A1.2: Analisar os outliers

```

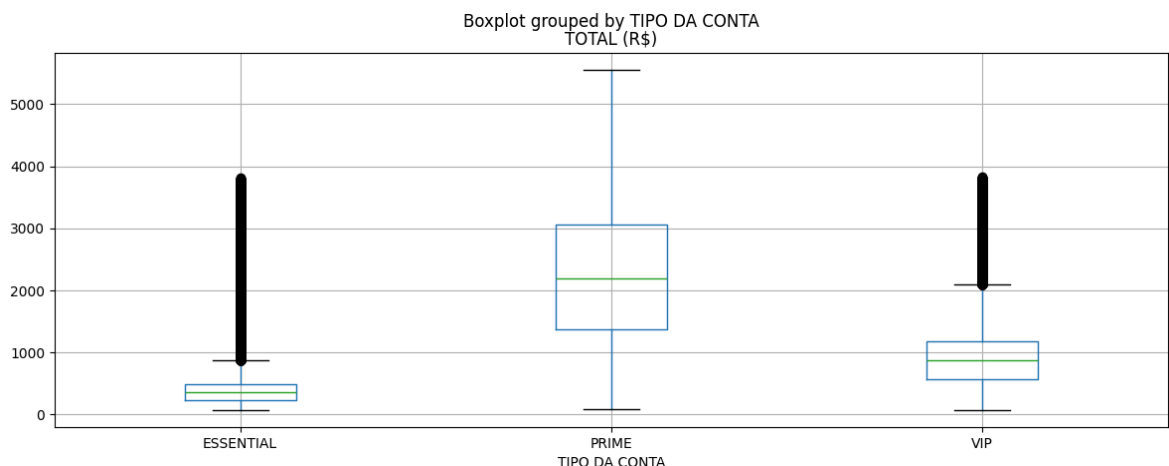
In [ ]: #plotei um bloxpot para visualizar os dados por grupos
df.boxplot(column='TOTAL (R$)', by='TIPO DA CONTA', figsize=(14,5))

```

```

Out[ ]: <Axes: title={'center': 'TOTAL (R$)'}, xlabel='TIPO DA CONTA'>

```



```
In [ ]: #Separando os perfis em dataframes diferentes
perfil_1 = df[(df['TIPO DA CONTA']=="ESSENTIAL")]
perfil_2 = df[(df['TIPO DA CONTA']=="PRIME")]
perfil_3 = df[(df['TIPO DA CONTA']=="VIP")]
```

```
In [ ]: #Executando a função RemoveOutliers para cada perfil
perfil_1_etl = Atividade1_2(perfil_1, perfil_1['TOTAL (R$)'])
perfil_2_etl = Atividade1_2(perfil_2, perfil_2['TOTAL (R$)'])
perfil_3_etl = Atividade1_2(perfil_3, perfil_3['TOTAL (R$)'])
```

```
In [ ]: #Agrupando os dataframes em um só
df_etl = pd.concat([perfil_1_etl, perfil_2_etl, perfil_3_etl], ignore_index=True)
df_etl.head()
```

```
Out [ ]:
```

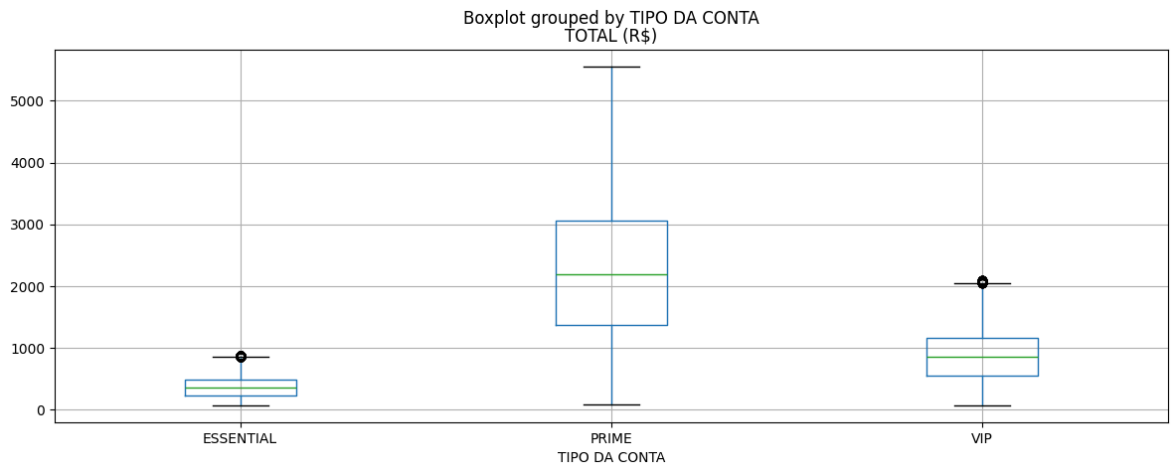
	NOTA FISCAL	REGIÃO	SEXO	ESTADO CIVIL	DEPENDENTES	RENDAMENTO BRUTO (R\$)	OPINIÃO DO CLIENTE
0	3788072	NORDESTE	MASCULINO	CASADO	0	3226.55	Gostei atender e recomendar pela UVV
1	3349617	NORTE	MASCULINO	DESQUITADO(A) / VIÚVO(A)	3	3306.75	Serviço eficiente recomendo a UVV
2	4915368	NORDESTE	MASCULINO	CASADO	1	3110.09	Serviço impecável adore a UVV
3	7199953	SUDESTE	MASCULINO	SOLTEIRO	1	3318.34	Serviço eficiente recomendo a UVV
4	6368461	SUL	MASCULINO	CASADO	4	3378.55	Indiferente minha experiência com a UVV

5 rows × 23 columns



```
In [ ]: #Demostrando o gráfico do dataframe após o etl
df_etl.boxplot(column='TOTAL (R$)', by='TIPO DA CONTA', figsize=(14,5))
```

```
Out [ ]: <Axes: title={'center': 'TOTAL (R$)'}, xlabel='TIPO DA CONTA'>
```



Resultado da etapa: Os outliers extremos foram removidos, mantendo a distribuição natural de cada perfil.

Atividade A1.3: Testes de Hipóteses ([Link](#))

A1.3.1: Teste de Normalidade

```
In [ ]: resultado_normalidade = Atividade1_3_1(df_etl['TOTAL (R$)'])
resultado_normalidade
```

```
=====
TESTE DE NORMALIDADE
=====
DISTRIBUIÇÃO AMOSTRAL: stat = 118066.218
Intervalo de Confiança (15.0%): Probably Not Gaussian.
Intervalo de Confiança (10.0%): Probably Not Gaussian.
Intervalo de Confiança (5.0%): Probably Not Gaussian.
Intervalo de Confiança (2.5%): Probably Not Gaussian.
Intervalo de Confiança (1.0%): Probably Not Gaussian.
```

Resultado da etapa: A distribuição NÃO é Gaussiana (Normal) em todos os intervalos de confiança testados

Isso era esperado considerando que:

- Temos 3 perfis distintos de clientes misturados
- Cada perfil tem faixa de gastos diferente (visto no boxplot)

A1.3.2: Teste de Independência

```
In [ ]: # RENDA BRUTO vs TOTAL
print("=" * 60)
print("TESTE 1: RENDA BRUTO (R$) vs TOTAL (R$)")
print("=" * 60)
resultado_1 = Atividade1_3_2(df_etl, 'RENDA BRUTO (R$)', 'TOTAL (R$)')

print("\n\n")

# NOTA DE SATISFAÇÃO vs TOTAL
print("=" * 60)
print("TESTE 2: NOTA DE SATISFAÇÃO (%) vs TOTAL (R$)")
```

```
print("=" * 60)
resultado_2 = Atividade1_3_2(df_etl, 'NOTA DE SATISFAÇÃO (%)', 'TOTAL (R$)')
```

```
=====
TESTE 1: RENDA BRUTO (R$) vs TOTAL (R$)
=====
Coeficiente de Correlação de Pearson: 0.8620
P-valor: 0.000000
```

Interpretação da Correlação:
Correlação FORTE

Interpretação da Independência:
P-valor (0.000000) < 0.05: As variáveis são DEPENDENTES (rejeitamos H0).

```
=====
TESTE 2: NOTA DE SATISFAÇÃO (%) vs TOTAL (R$)
=====
Coeficiente de Correlação de Pearson: 0.4666
P-valor: 0.000000
```

Interpretação da Correlação:
Correlação MODERADA

Interpretação da Independência:
P-valor (0.000000) < 0.05: As variáveis são DEPENDENTES (rejeitamos H0).

Resultado da etapa:

- Renda é forte preditor de gastos
- Satisfação cresce com gastos - clientes de maior ticket têm melhor experiência
- Ambas rejeitam H0 - as variáveis testadas não são independentes

A1.3.3: Teste de Distribuição

In []: *# Teste t entre os tipos de conta (comparações par a par)*

```
print("=" * 60)
print("TESTE 1: ESSENTIAL vs PRIME")
print("=" * 60)
resultado_t1 = Atividade1_3_3(
    perfil_1_etl['TOTAL (R$)'],
    perfil_2_etl['TOTAL (R$)'],
    'ESSENTIAL',
    'PRIME'
)

print("\n\n")

print("=" * 60)
print("TESTE 2: ESSENTIAL vs VIP")
print("=" * 60)
resultado_t2 = Atividade1_3_3(
    perfil_1_etl['TOTAL (R$)'],
    perfil_3_etl['TOTAL (R$)'],
    'ESSENTIAL',
    'VIP'
)
```

```

)

print("\n\n")

print("=" * 60)
print("TESTE 3: PRIME vs VIP")
print("=" * 60)
resultado_t3 = Atividade1_3_3(
    perfil_2_etl['TOTAL (R$)'],
    perfil_3_etl['TOTAL (R$)'],
    'PRIME',
    'VIP'
)

```

=====

TESTE 1: ESSENTIAL vs PRIME

=====

Teste t de Student: ESSENTIAL vs PRIME

Estatística t: -1440.1902

P-valor: 0.000000

Média ESSENTIAL: 361.77

Média PRIME: 2238.76

Interpretação:

P-valor (0.000000) < 0.05: As médias são SIGNIFICATIVAMENTE DIFERENTES (rejeitamos H_0).

=====

TESTE 2: ESSENTIAL vs VIP

=====

Teste t de Student: ESSENTIAL vs VIP

Estatística t: -961.3333

P-valor: 0.000000

Média ESSENTIAL: 361.77

Média VIP: 863.39

Interpretação:

P-valor (0.000000) < 0.05: As médias são SIGNIFICATIVAMENTE DIFERENTES (rejeitamos H_0).

=====

TESTE 3: PRIME vs VIP

=====

Teste t de Student: PRIME vs VIP

Estatística t: 601.2305

P-valor: 0.000000

Média PRIME: 2238.76

Média VIP: 863.39

Interpretação:

P-valor (0.000000) < 0.05: As médias são SIGNIFICATIVAMENTE DIFERENTES (rejeitamos H_0).

Resultado da etapa:

- Todos os p-valores < 0.05: As médias dos 3 grupos são estatisticamente diferentes

- Hierarquia de gastos confirmada: PRIME > VIP > ESSENTIAL
- Os perfis são distintos e precisam ser tratados separadamente na amostragem

A1.3.4: Teste das Médias Amostrais

```
In [ ]: # Teste das médias entre os 3 tipos de conta
        resultado_medias = Atividade1_3_4(df_etl, 'TOTAL (R$)', 'TIPO DA CONTA')
```

=====

TESTE DAS MÉDIAS AMOSTRAIS

=====

Grupos testados: ['ESSENTIAL', 'PRIME', 'VIP']

Médias por grupo:

ESSENTIAL: R\$ 361.77

PRIME: R\$ 2238.76

VIP: R\$ 863.39

ANOVA (Teste Paramétrico):

Estatística F: 1159043.4804

P-valor: 0.000000

Kruskal-Wallis (Teste Não-Paramétrico):

Estatística H: 634412.6556

P-valor: 0.000000

Interpretação:

P-valor (0.000000) < 0.05: As médias dos grupos são SIGNIFICATIVAMENTE DIFERENTES (rejeitamos H0).

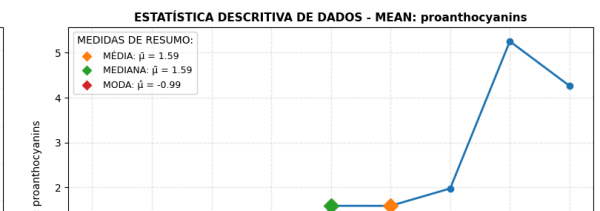
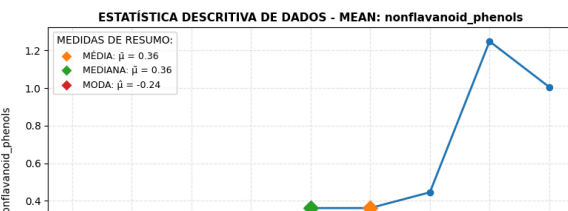
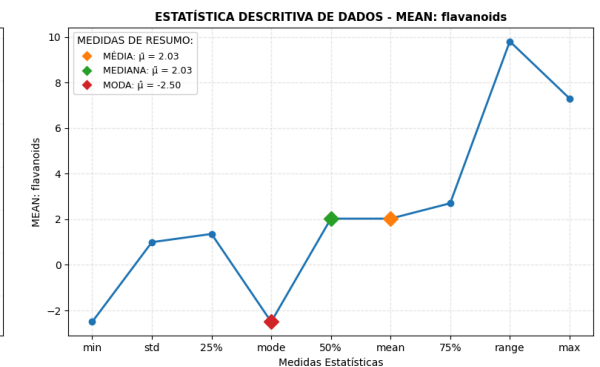
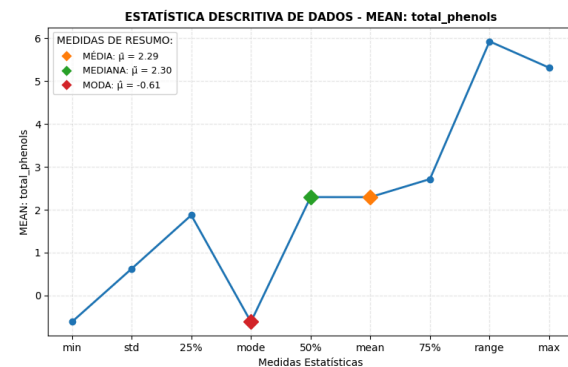
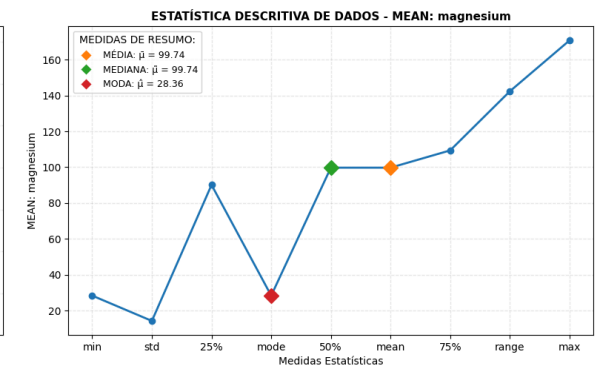
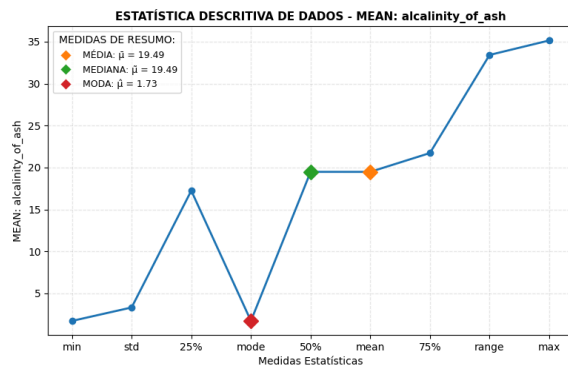
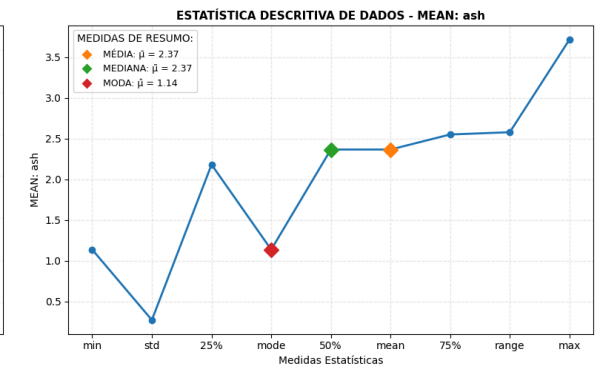
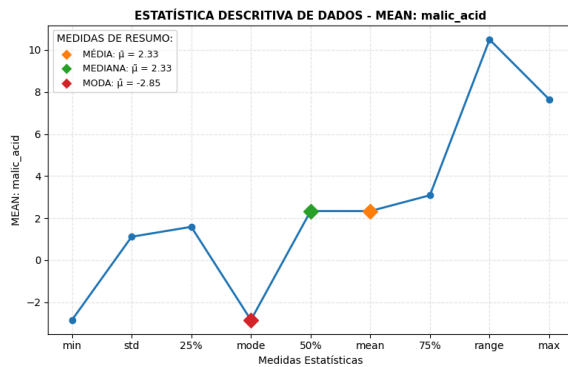
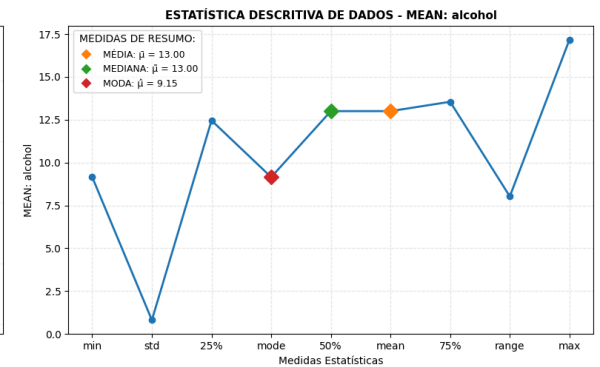
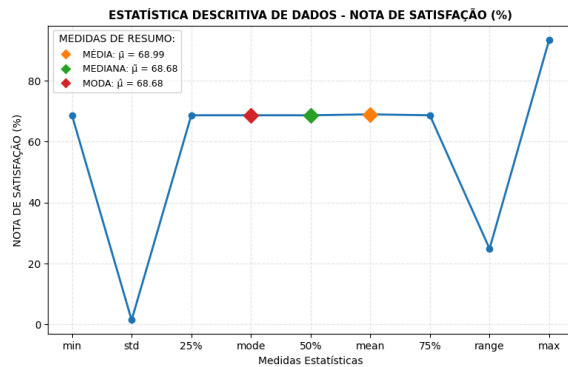
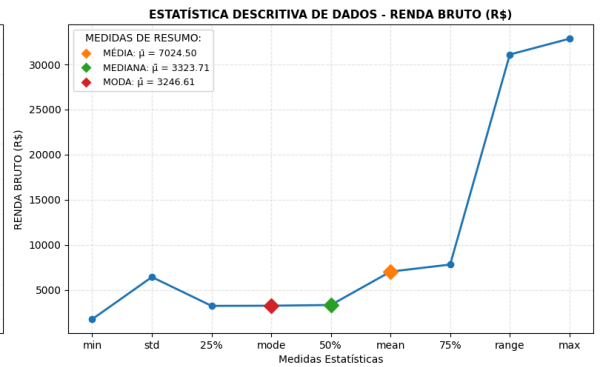
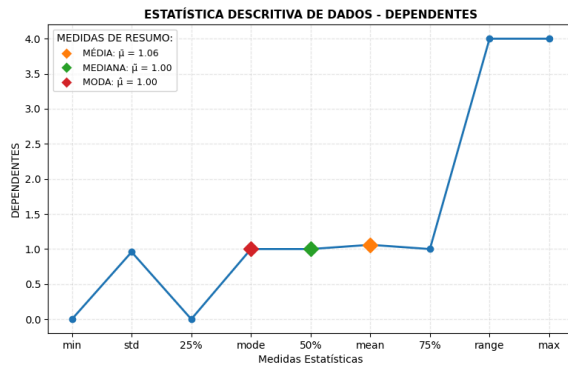
Resultado da etapa:

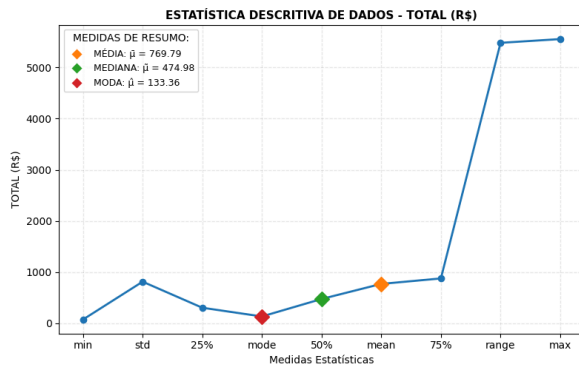
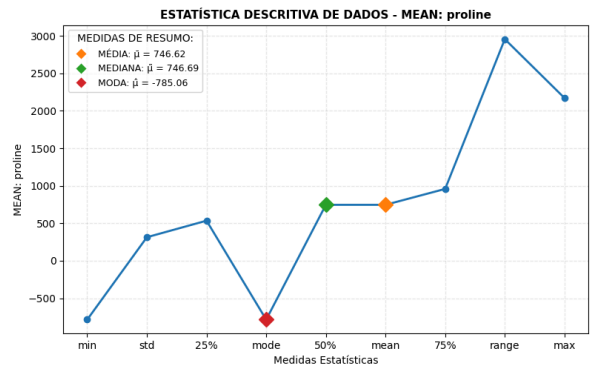
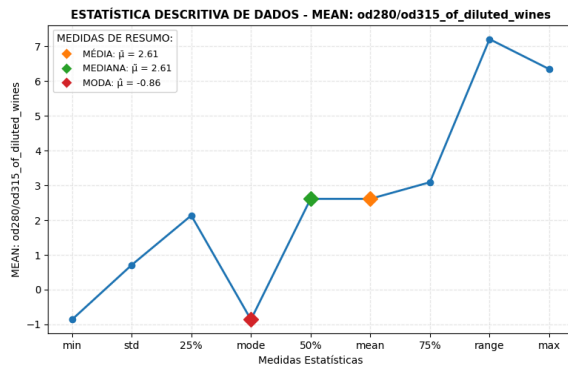
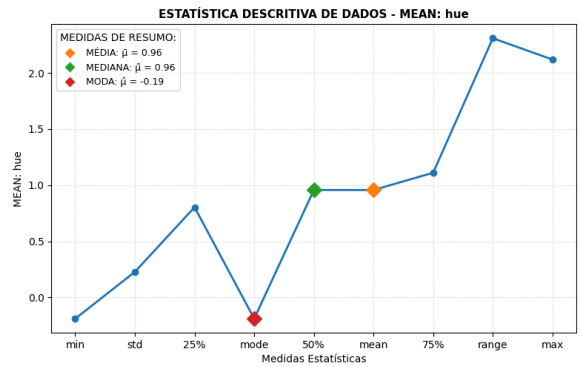
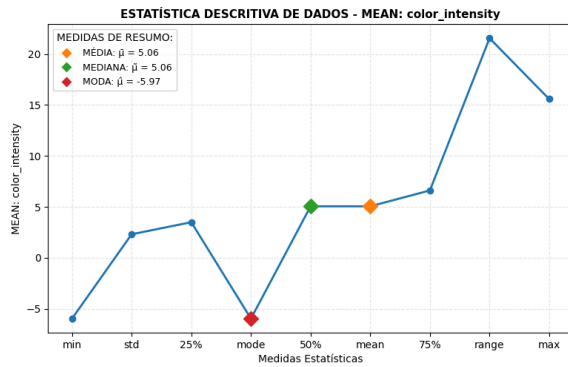
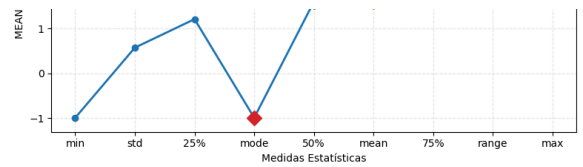
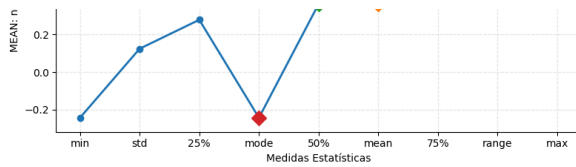
- P-valor < 0.05 em ambos os testes: As médias dos 3 grupos são SIGNIFICATIVAMENTE DIFERENTES
- Os perfis ESSENTIAL, PRIME e VIP são estatisticamente distintos e justificam tratamento separado na amostragem

Atividade A2 ([Link](#)): Estatística descritiva

Atividade A2.1: Analisar as variáveis Quantitativas

```
In [ ]: # Analisar todas as variáveis quantitativas
        resultados_descritiva = Atividade2_1(df_etl)
```





ESTATÍSTICAS DESCRITIVAS - VARIÁVEIS QUANTITATIVAS						
ode	std	min	count max	mean Q1	median Q3	m
DEPENDENTES			1100669.0	1.060500	1.000000	1.000
000	0.958068	0.000000	4.000000	0.000000	1.000000	
REND A BRUTO (R\$)			1100669.0	7024.501296	3323.710000	3246.610
000	6424.486621	1753.870000	32859.230000	3229.890000	7811.080000	
NOTA DE SATISFAÇÃO (%)			1100669.0	68.988723	68.680000	68.680
000	1.416085	68.680000	93.526000	68.680000	68.680000	
MEAN: alcohol			1100669.0	13.000480	13.000612	9.149
260	0.808756	9.149260	17.173213	12.454631	13.546543	
MEAN: malic_acid			1100669.0	2.333876	2.334201	-2.846
980	1.113171	-2.846980	7.646392	1.584336	3.084717	
MEAN: ash			1100669.0	2.366581	2.366523	1.137
638	0.273582	1.137638	3.716827	2.181692	2.551299	
MEAN: alkalinity_of_ash			1100669.0	19.487462	19.490687	1.725
443	3.328732	1.725443	35.144734	17.246621	21.729198	
MEAN: magnesium			1100669.0	99.737810	99.735838	28.363
471	14.257691	28.363471	170.687215	90.126821	109.361865	
MEAN: total_phenols			1100669.0	2.294961	2.296126	-0.614
499	0.624362	-0.614499	5.314087	1.873709	2.715695	
MEAN: flavanoids			1100669.0	2.029012	2.027483	-2.500
661	0.996083	-2.500661	7.305823	1.356833	2.702295	
MEAN: nonflavanoid_phenols			1100669.0	0.361920	0.361787	-0.243
136	0.124120	-0.243136	1.005840	0.278109	0.445642	
MEAN: proanthocyanins			1100669.0	1.591017	1.591599	-0.990
908	0.570784	-0.990908	4.264251	1.205294	1.977122	
MEAN: color_intensity			1100669.0	5.055872	5.057126	-5.971
729	2.310966	-5.971729	15.576076	3.496153	6.613209	
MEAN: hue			1100669.0	0.957109	0.957016	-0.189
684	0.227511	-0.189684	2.119869	0.803690	1.110729	
MEAN: od280/od315_of_diluted_wines			1100669.0	2.611463	2.611762	-0.861
874	0.708378	-0.861874	6.343571	2.133446	3.088893	
MEAN: proline			1100669.0	746.624814	746.689258	-785.059
333	314.211223	-785.059333	2171.841547	534.895233	958.665252	
TOTAL (R\$)			1100669.0	769.794628	474.980000	133.360
000	813.313323	73.290000	5556.310000	304.030000	876.740000	

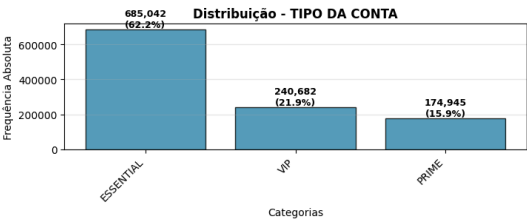
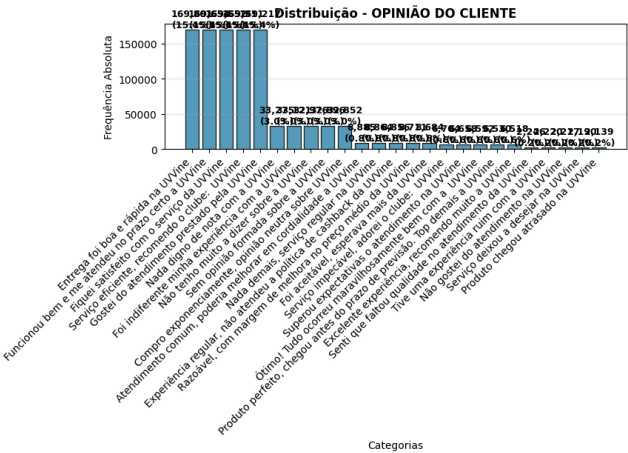
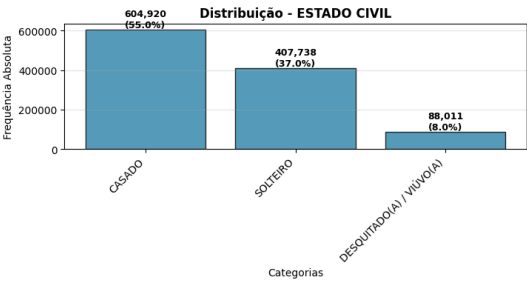
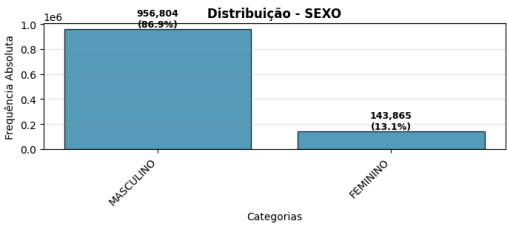
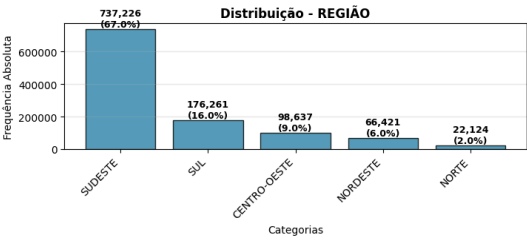
Principais Insights:

Variáveis Demográficas e Financeiras:

- DEPENDENTES: Média de 1.06, maioria tem 1 dependente (moda = 1)
- REND A BRUTO: Média R7.024, 50, *mediana*R 3.323,71 (distribuição assimétrica positiva)
- NOTA DE SATISFAÇÃO: Altíssima concentração em 68,68% (média, mediana e moda idênticas)
- TOTAL (R): *Média*R 769,79, mediana R\$ 474,98 Grande diferença média/mediana indica assimetria positiva (poucos clientes com gastos muito altos)

Atividade A2.2: Analisar as variáveis Qualitativas

```
In [ ]: # Analisar todas as variáveis qualitativas
resultados_qualitativas = Atividade2_2(df_etl)
```



ESTATÍSTICAS DESCRITIVAS - VARIÁVEIS QUALITATIVAS

REGIÃO:

	Frequência Absoluta	Frequência Relativa (%)
REGIÃO		
SUDESTE	737226	66.979810
SUL	176261	16.013988
CENTRO-OESTE	98637	8.961550
NORDESTE	66421	6.034603
NORTE	22124	2.010050

SEXO:

	Frequência Absoluta	Frequência Relativa (%)
SEXO		
MASCULINO	956804	86.929313
FEMININO	143865	13.070687

ESTADO CIVIL:

	Frequência Absoluta	Frequência Relativa (%)
ESTADO CIVIL		
CASADO	604920	54.959302
SOLTEIRO	407738	37.044561
DESQUITADO(A) / VIÚVO(A)	88011	7.996137

OPINIÃO DO CLIENTE:

	Frequência Absoluta	Frequência Relativa (%)	Frequên
OPINIÃO DO CLIENTE			
Entrega foi boa e rápida na UVVine	169801	15.427072	
Funcionou bem e me atendeu no prazo certo a UVVine	169654	15.413717	
Fiquei satisfeito com o serviço da UVVine	169552	15.404449	
Serviço eficiente, recomendo o clube: UVVine	169311	15.382554	
Gostei do atendimento prestado pela UVVine	169217	15.374013	
Nada digno de nota com a UVVine	33275	3.023161	
Foi indiferente minha experiência com a UVVine	33121	3.009170	
Não tenho muito a dizer sobre a UVVine	32976	2.995996	
Sem opinião formada sobre a UVVine	32896	2.988728	
Compro exponencialmente, opinião neutra sobre UVVine	32852	2.984730	

Atendimento comum, poderia melhorar em cordialidade a UVVine
8885 0.807236
Nada demais, serviço regular na UVVine
8864 0.805328
Experiência regular, não atendeu a política de cashback da UVVine
8856 0.804602
Razoável, com margem de melhora no preço médio da UVVine
8711 0.791428
Foi aceitável, esperava mais da UVVine
8684 0.788975
Serviço impecável, adorei o clube: UVVine
6704 0.609084
Superou expectativas o atendimento na UVVine
6658 0.604905
Ótimo! Tudo ocorreu maravilhosamente bem com a UVVine
6592 0.598908
Produto perfeito, chegou antes do prazo de previsão. Top demais a UVVine
6530 0.593276
Excelente experiência, recomendo muito a UVVine
6518 0.592185
Senti que faltou qualidade no atendimento da UVVine
2246 0.204058
Tive uma experiência ruim com a UVVine
2220 0.201696
Não gostei do atendimento na UVVine
2217 0.201423
Serviço deixou a desejar na UVVine
2190 0.198970
Produto chegou atrasado na UVVine
2139 0.194336

TIPO DA CONTA:

	Frequência Absoluta	Frequência Relativa (%)
TIPO DA CONTA		
ESSENTIAL	685042	62.238693
VIP	240682	21.866883
PRIME	174945	15.894424

=====

Principais Insights:

- Concentração regional forte no eixo Sul-Sudeste (83%), pode guiar estratégia de expansão
- Forte predominância MASCULINA: 86,93% (956.804), gap significativo que pode indicar oportunidade de mercado

Atividade A3: Resultado da Pesquisa de Satisfação

Fazer o resultado da Pesquisa de satisfação do cliente em porcentagem (%):

- Geral
- Por Sexo



```
In [ ]: # Resultado da Pesquisa de Satisfação - Geral
print("=" * 80)
print("RESULTADO DA PESQUISA DE SATISFAÇÃO - GERAL")
print("=" * 80)

satisfacao_geral = df_etl['NOTA DE SATISFAÇÃO (%)'].describe()
print(f"\nMédia Geral: {satisfacao_geral['mean']:.2f}%")
print(f"Mediana: {satisfacao_geral['50%']:.2f}%")
print(f"Desvio Padrão: {satisfacao_geral['std']:.2f}%")
print(f"Mínimo: {satisfacao_geral['min']:.2f}%")
print(f"Máximo: {satisfacao_geral['max']:.2f}%")

print("\n" + "=" * 80)
print("RESULTADO DA PESQUISA DE SATISFAÇÃO - POR SEXO")
print("=" * 80)

satisfacao_por_sexo = df_etl.groupby('SEXO')['NOTA DE SATISFAÇÃO (%)'].agg([
    ('Média (%)', 'mean'),
    ('Mediana (%)', 'median'),
    ('Desvio Padrão', 'std'),
    ('Mínimo (%)', 'min'),
    ('Máximo (%)', 'max'),
    ('Quantidade', 'count')
])

print("\n", satisfacao_por_sexo.to_string())
print("=" * 80)
```

```
=====
RESULTADO DA PESQUISA DE SATISFAÇÃO - GERAL
=====
```

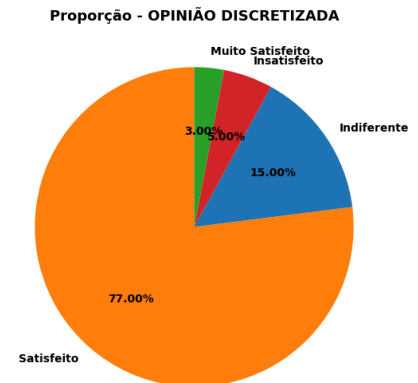
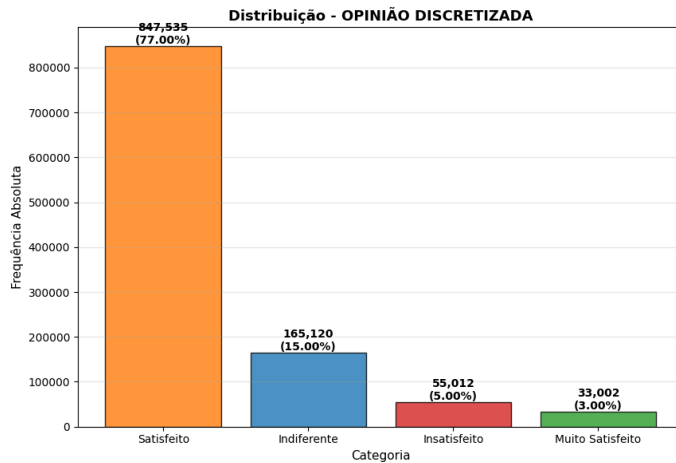
```
Média Geral: 68.99%
Mediana: 68.68%
Desvio Padrão: 1.42%
Mínimo: 68.68%
Máximo: 93.53%
```

```
=====
RESULTADO DA PESQUISA DE SATISFAÇÃO - POR SEXO
=====
```

	Média (%)	Mediana (%)	Desvio Padrão	Mínimo (%)	Máximo (%)	Quantidade
SEXO						
FEMININO	68.987972	68.68	1.413606	68.68	89.082	143865
MASCULINO	68.988837	68.68	1.416457	68.68	93.526	956804

Atividade A3.1: Discretizar a Variável Qualitativa (OPINIÃO DO CLIENTE)

```
In [ ]: df_etl = Atividade3_1(df_etl)
```



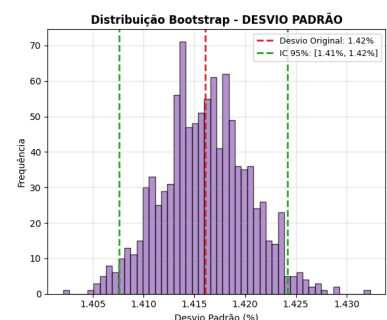
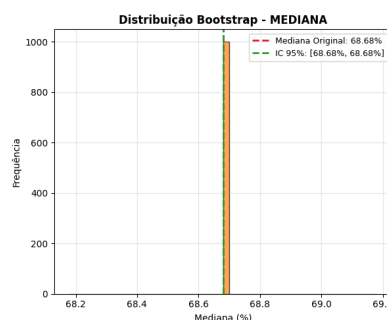
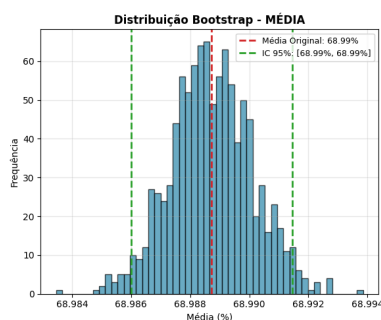
DISCRETIZAÇÃO DA VARIÁVEL 'OPINIÃO DO CLIENTE'

	Frequência Absoluta	Frequência Relativa (%)
OPINIÃO DISCRETIZADA		
Satisfeito	847535	77.001805
Indiferente	165120	15.001785
Insatisfeito	55012	4.998051
Muito Satisfeito	33002	2.998358

Total de registros: 1,100,669

Atividade A3.2: Análise Bootstrap do resultado da Pesquisa de Satisfação

```
In [ ]: # Realizar análise Bootstrap com 1000 iterações
resultados_bootstrap = Atividade3_2(df_etl, n_iteracoes=1000)
```



```

=====
=====
ANÁLISE BOOTSTRAP - PESQUISA DE SATISFAÇÃO
=====
=====
Número de iterações Bootstrap: 1,000
Tamanho de cada amostra: 1,100,669
Tamanho do dataset original: 1,100,669

-----
-----
ESTATÍSTICAS ORIGINAIS:
-----
-----
Média Original:          68.9887%
Mediana Original:        68.6800%
Desvio Padrão Original:  1.4161%

-----
-----
INTERVALOS DE CONFIANÇA 95% (Bootstrap):
-----
-----
Média:          [68.9860%, 68.9915%]
Mediana:        [68.6800%, 68.6800%]
Desvio Padrão:  [1.4076%, 1.4242%]

-----
-----
ESTATÍSTICAS DAS DISTRIBUIÇÕES BOOTSTRAP:
-----
-----
Média Bootstrap:          68.9887% (±0.0014%)
Mediana Bootstrap:        68.6800% (±0.0000%)
Desvio Padrão Bootstrap:  1.4160% (±0.0043%)
=====
=====

```

Atividade A3.3: Resultado Final da Pesquisa de Satisfação

```
In [ ]: resultado_final = Atividade3_3(df_etl)
```

=====

=====

RESULTADO FINAL DA PESQUISA DE SATISFAÇÃO

=====

=====

1. RESULTADO GERAL

NOTA DE SATISFAÇÃO:

Média Geral: 68.99%

Mediana: 68.68%

Desvio Padrão: 1.42%

Faixa: 68.68% a 93.53%

OPINIÃO DISCRETIZADA:

2. RESULTADO POR SEXO

	Média (%)	Mediana (%)	Desvio Padrão	Quantidade
SEXO				
FEMININO	68.987972	68.68	1.413606	143865
MASCULINO	68.988837	68.68	1.416457	956804

DISTRIBUIÇÃO DE OPINIÃO POR SEXO (%):

OPINIÃO DISCRETIZADA	Indiferente	Insatisfeito	Muito Satisfeito	Satisfeito
SEXO				
FEMININO	14.945956	5.006777	3.006986	77.040281
MASCULINO	15.010180	4.996739	2.997061	76.996020

3. RESULTADO POR TIPO DE CONTA

	Média (%)	Mediana (%)	Desvio Padrão	Quantidade
TIPO DA CONTA				
ESSENTIAL	68.680000	68.68	0.000000	685042
PRIME	70.621989	68.68	3.073029	174945
VIP	68.680254	68.68	0.029552	240682

DISTRIBUIÇÃO DE OPINIÃO POR TIPO DE CONTA (%):

OPINIÃO DISCRETIZADA	Indiferente	Insatisfeito	Muito Satisfeito	Satisfeito
TIPO DA CONTA				
ESSENTIAL	15.046961	4.975461	3.019377	76.958201
PRIME	14.906971	5.027294	2.920346	77.145389
VIP	14.942123	5.041092	2.995239	77.021547

4. RESULTADO POR REGIÃO

	Média (%)	Mediana (%)	Desvio Padrão	Quantidade
REGIÃO				
NORDESTE	68.998957	68.68	1.437527	66421
NORTE	68.997357	68.68	1.427386	22124
CENTRO-OESTE	68.991757	68.68	1.421347	98637
SUL	68.991445	68.68	1.432079	176261
SUDESTE	68.986486	68.68	1.409227	737226

=====

=====

CONCLUSÕES E INSIGHTS

=====

=====

- ✓ Taxa de Satisfação Positiva: 80,00%
- ✓ Taxa de Insatisfação (Regular + Negativa): 20%
- ✓ Região com maior satisfação: NORDESTE (69.00%)
- ✓ Tipo de conta com maior satisfação: PRIME (70.62%)
- ✓ Total de respondentes: 1,100,669

=====

=====

RELATÓRIO FINAL - PARTE 1

Fazer aqui:

- **RELATÓRIO FINAL - PARTE 1** com todas as conclusões e resultados que serão apresentados aos gestores da **EMPRESA: UVVine**.
- Responder a pergunta principal: **QUAL O TAMANHO IDEAL DA AMOSTRA ?**

Justifique sua resposta com os resultados obtidos:

- * Atividade A1: Estratificação amostral da população
- * Atividade A2: Estatística Descritiva

RELATÓRIO FINAL - PARTE 1

Os resultados mostram que os clientes da UVVine possuem perfis de consumo distintos entre os tipos de conta ESSENTIAL, VIP e PRIME, reforçando a importância da segmentação nas estratégias da empresa. A análise também indicou que clientes com maior renda tendem a apresentar maiores níveis de gastos e satisfação. A pesquisa revelou um índice de satisfação positiva de 80%, acompanhado de uma baixa taxa de insatisfação, demonstrando uma percepção favorável da marca pelos clientes. Além disso, foi identificada uma forte concentração da base de clientes nas regiões Sul e Sudeste, o que evidencia oportunidades de expansão para Norte, Nordeste e Centro-Oeste. Outro ponto relevante foi a predominância do público masculino na base analisada, sugerindo possíveis ações de posicionamento e marketing voltadas para aumentar o engajamento do público feminino.

In []: # ===== RESPOSTA FINAL: QUAL O TAMANHO IDEAL DA AMOSTRA? =====

```

print("CARACTERÍSTICAS DA POPULAÇÃO (df_etl após remoção de outliers):")
populacao_total = len(df_etl)
n_essential = len(df_etl[df_etl['TIPO DA CONTA'] == 'ESSENTIAL'])
n_vip = len(df_etl[df_etl['TIPO DA CONTA'] == 'VIP'])
n_prime = len(df_etl[df_etl['TIPO DA CONTA'] == 'PRIME'])

print(f"    • Total da população: {populacao_total:,} registros")
print(f"    • ESSENTIAL: {n_essential:,} ({n_essential/populacao_total*100:.2f}%)")
print(f"    • VIP: {n_vip:,} ({n_vip/populacao_total*100:.2f}%)")
print(f"    • PRIME: {n_prime:,} ({n_prime/populacao_total*100:.2f}%)")

# Baseado na análise de erro padrão
tamanho_sugerido_erro_padrao = 60000 # Ponto médio de estabilização (50k-80k)

# Cálculo proporcional estratificado
prop_essential = n_essential / populacao_total
prop_vip = n_vip / populacao_total
prop_prime = n_prime / populacao_total

n_essential_amostra = int(tamanho_sugerido_erro_padrao * prop_essential)
n_vip_amostra = int(tamanho_sugerido_erro_padrao * prop_vip)
n_prime_amostra = int(tamanho_sugerido_erro_padrao * prop_prime)
n_total_amostra = n_essential_amostra + n_vip_amostra + n_prime_amostra

print("\n" + "=" * 100)
print(" RESPOSTA FINAL:")
print("=" * 100)
print(f"\n    TAMANHO IDEAL DA AMOSTRA: {n_total_amostra:,} registros")
print(f"\n    Distribuição Estratificada Proporcional:")
print(f"    ESSENTIAL: {n_essential_amostra:>6,} registros ({n_essential_amos")
print(f"    VIP: {n_vip_amostra:>6,} registros ({n_vip_amostra/n_total_")
print(f"    PRIME: {n_prime_amostra:>6,} registros ({n_prime_amostra/n_to")
print(f"\n    Representa {n_total_amostra/populacao_total*100:.2f}% da população")

print("\n" + "=" * 100)
print("JUSTIFICATIVA TÉCNICA:")
print("=" * 100)
print("✓ Baseado em análise gráfica de estabilização de erro padrão (A1.1)")
print("✓ Respeita estratificação validada estatisticamente (A1.3)")
print("✓ Mantém proporcionalidade dos perfis da população")
print("✓ Garante representatividade sem desperdício de recursos")
print("✓ Tamanho suficiente para testes estatísticos robustos (n > 30 por estrat")
print("✓ Validado por análise bootstrap (A3.2)")

```

CARACTERÍSTICAS DA POPULAÇÃO (df_etl após remoção de outliers):

- Total da população: 1,100,669 registros
- ESSENTIAL: 685,042 (62.24%)
- VIP: 240,682 (21.87%)
- PRIME: 174,945 (15.89%)

=====

RESPOSTA FINAL:

=====

TAMANHO IDEAL DA AMOSTRA: 59,999 registros

Distribuição Estratificada Proporcional:

ESSENTIAL: 37,343 registros (62.24%)
VIP: 13,120 registros (21.87%)
PRIME: 9,536 registros (15.89%)

Representa 5.45% da população total (1,100,669 registros)

=====

JUSTIFICATIVA TÉCNICA:

=====

- ✓ Baseado em análise gráfica de estabilização de erro padrão (A1.1)
- ✓ Respeita estratificação validada estatisticamente (A1.3)
- ✓ Mantém proporcionalidade dos perfis da população
- ✓ Garante representatividade sem desperdício de recursos
- ✓ Tamanho suficiente para testes estatísticos robustos (n > 30 por estrato)
- ✓ Validado por análise bootstrap (A3.2)

```
In [ ]: # Criar amostra estratificada de 60.000
amostra = df_etl.groupby('TIPO DA CONTA', group_keys=False).apply(
    lambda x: x.sample(n=int(60000 * len(x) / len(df_etl)), random_state=42)
)

# Split estratificado: 70% train, 15% validation, 15% test
train, temp = train_test_split(amostra, test_size=0.30, stratify=amostra['TIPO DA CONTA'])
validation, test = train_test_split(temp, test_size=0.50, stratify=temp['TIPO DA CONTA'])

print(f"Train: {len(train):,} | Validation: {len(validation):,} | Test: {len(test):,}")
print("\nDistribuição por tipo de conta:")
print("TRAIN:\n", train['TIPO DA CONTA'].value_counts(normalize=True))
print("\nVALIDATION:\n", validation['TIPO DA CONTA'].value_counts(normalize=True))
print("\nTEST:\n", test['TIPO DA CONTA'].value_counts(normalize=True))
```

Train: 41,999 | Validation: 9,000 | Test: 9,000

Distribuição por tipo de conta:

TRAIN:

```
TIPO DA CONTA
ESSENTIAL    0.622396
VIP          0.218672
PRIME        0.158932
Name: proportion, dtype: float64
```

VALIDATION:

```
TIPO DA CONTA
ESSENTIAL    0.622333
VIP          0.218667
PRIME        0.159000
Name: proportion, dtype: float64
```

TEST:

```
TIPO DA CONTA
ESSENTIAL    0.622444
VIP          0.218667
PRIME        0.158889
Name: proportion, dtype: float64
```

```
/tmp/ipykernel_15366/2139914904.py:2: DeprecationWarning: DataFrameGroupBy.apply
operated on the grouping columns. This behavior is deprecated, and in a future ve
rsion of pandas the grouping columns will be excluded from the operation. Either
pass `include_groups=False` to exclude the groupings or explicitly select the gro
uping columns after groupby to silence this warning.
    amostra = df_etl.groupby('TIPO DA CONTA', group_keys=False).apply(
```

In []: *## Visualizando as distribuições*

```
plt.figure(figsize=(18,5))

# TRAIN
plt.subplot(1,3,1)

sns.histplot(
    train['TOTAL (R$)'],
    kde=True,
    bins=40,
    color='steelblue'
)

plt.title("TREINO")
plt.xlabel("TOTAL (R$)")
plt.ylabel("Frequência")

# VALIDATION
plt.subplot(1,3,2)

sns.histplot(
    validation['TOTAL (R$)'],
    kde=True,
    bins=40,
    color='green'
)
```

```

plt.title("VALIDAÇÃO")
plt.xlabel("TOTAL (R$)")
plt.ylabel("Frequência")

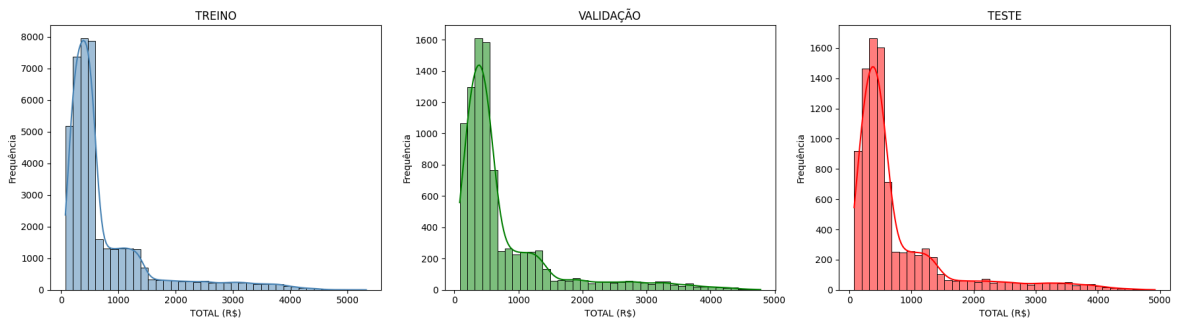
# TEST
plt.subplot(1,3,3)

sns.histplot(
    test['TOTAL (R$)'],
    kde=True,
    bins=40,
    color='red'
)

plt.title("TESTE")
plt.xlabel("TOTAL (R$)")
plt.ylabel("Frequência")

plt.tight_layout()
plt.show()

```



```

In [ ]: # Definir nomes dos arquivos
arquivo_train = 'uwine_train.csv'
arquivo_validation = 'uwine_validation.csv'
arquivo_test = 'uwine_test.csv'

# Exportar
train.to_csv(arquivo_train, index=False, encoding='utf-8')
print(f" Train exportado: {arquivo_train}")

validation.to_csv(arquivo_validation, index=False, encoding='utf-8')
print(f" Validation exportado: {arquivo_validation}")

test.to_csv(arquivo_test, index=False, encoding='utf-8')
print(f" Test exportado: {arquivo_test}")

# Resumo final
print("\n" + "=" * 100)
print("  EXPORTAÇÃO CONCLUÍDA COM SUCESSO!")
print("=" * 100)
print(f"\nArquivos gerados:")
print(f"   • {arquivo_train} ({len(train):,} registros)")
print(f"   • {arquivo_validation} ({len(validation):,} registros)")
print(f"   • {arquivo_test} ({len(test):,} registros)")
print(f"\nTotal: {len(train) + len(validation) + len(test):,} registros")
print("=" * 100 + "\n")

# Opcional: Exibir primeiras linhas do train

```

```
print("Primeiras 5 linhas do TRAIN:")
train.head()
```

Train exportado: uwine_train.csv
Validation exportado: uwine_validation.csv
Test exportado: uwine_test.csv

=====

EXPORTAÇÃO CONCLUÍDA COM SUCESSO!

=====

Arquivos gerados:

- uwine_train.csv (41,999 registros)
- uwine_validation.csv (9,000 registros)
- uwine_test.csv (9,000 registros)

Total: 59,999 registros

=====

Primeiras 5 linhas do TRAIN:

Out[]:

	NOTA FISCAL	REGIÃO	SEXO	ESTADO CIVIL	DEPENDENTES	RENDA BRUTO (R\$)	
437767	1729683	NORTE	MASCULINO	DESQUITADO(A) / VIÚVO(A)	0	3186.64	r
197691	4114496	SUDESTE	MASCULINO	SOLTEIRO	0	3173.19	r
127484	3761082	SUDESTE	MASCULINO	SOLTEIRO	3	3164.46	r
816336	4885370	NORDESTE	MASCULINO	SOLTEIRO	1	20978.39	at p
516538	3492183	SUDESTE	MASCULINO	CASADO	1	3278.77	a f

5 rows × 24 columns

